Trace Equation Systems for Array Algorithms: Worked Examples

Gidon Ernst
LMU Munich, Germany
gidon.ernst@gmail.com

October 2026

These notes document the examples of the folder `examples/dafny/algorithms` of Cuvée in the style of the paper *Trace Equation Systems for Program Verification*: for each program we show the trace equation system that Cuvée generates (`-teq`), the lemmas that the simplifier applies (`-simplify`), how they are instantiated, and the conceptual idea behind them. All lemmas are stated in the Dafny dialect of Cuvée at the beginning of the respective file; the lemma database (the modules of `resources/cuvee/teq`) is injected by `-teq`, and the module of the fragments of inner loops by `-include teq:nested` where a file uses it. We write $\lfloor X \rfloor$ for `butlast(X)`, the trace X without its last element, $\mathsf{current}(X)$ for its last element, and $\mathsf{counter}(i, c, n)$ for the counter $\langle i, i+c, \ldots, i+nc \rangle$ of n steps. The sections are ordered by the complexity of the proof, from a single loop with two lemmas to loops in sequence and nested loops with summaries and inductions; Section 11 documents an attempt that is open. Sections 12 to 17 re-trace the proofs of some of the C programs of the MonoCera benchmarks with the database alone, and Section 18 collects the principles that the examples share, and Section 19 reads them as symptoms of the engine and draws the conclusion for the representation. Capital letters K, I, B, M denote traces, hatted letters $\hat{M}, \hat{J}, \hat{N}$ the fresh *sequences* that the summary of an inner loop introduces, one entry per iteration of the outer loop. $\mathsf{swap}(x, p, q)$ abbreviates the double store $x[p := x[q]][q := x[p]]$, which is how the swaps appear in the trace equations.

1 Linear Search: an Iteration Constraint Abstracted and Instantiated

1.1 The program

```
method lsearch(a: array<int, int>, n: int, x: int) returns (i: int)
  requires 0 <= n
  ensures 0 <= i <= n
  ensures i < n ==> a[i] == x
  ensures forall k: int :: 0 <= k < i ==> a[k] != x
{
  i := 0;
  while i < n && a[i] != x { i := i + 1; }
}
```

The first index of x among $a[0..n)$, or n. The loop test is data-dependent, so the number of iterations is not determined by n; the trace equation system records the test as the iteration constraint and its negation as the exit.

1.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad i = \mathsf{current}(I) \qquad I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ i, k \mapsto i + 1) \tag{1}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, i \mapsto i < n \wedge a[i] \neq x) \tag{2}$$

$$\neg(i < n \wedge a[i] \neq x) \tag{3}$$

$$\implies\ 0 \leq i \leq n\ \wedge\ (i < n \Rightarrow a[i] = x)\ \wedge\ (\forall k.\ 0 \leq k < i \Rightarrow a[k] \neq x)$$

With `-split`, the conclusion becomes four goals, one per conjunct, the last two with their premises as hypotheses: $i < n \Rightarrow a[i] = x$, and $0 \leq k < i \wedge a[k] = x \Rightarrow \mathsf{false}$. The simplifier's own rules do the following: I is a counter by counter_def, so $i = \mathsf{integer}(n')$; the constraint (2) is distributed into $\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n') \rfloor\ \mathsf{check}\ k \mapsto k < n)$, which invs_counter_upper resolves to $n' = 0 \vee \mathsf{integer}(n') \leq n$ (hence $0 \leq i \leq n$, the first two goals), and $\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n') \rfloor\ \mathsf{check}\ k \mapsto a[k] \neq x)$, the search constraint proper; the exit (3) is $n \leq i \vee a[i] = x$, which with $i < n$ gives the third goal.

1.3 The lemmas

The last goal needs the search constraint at a witness: for $0 \leq k < i$, the constraint says $a[k] \neq x$. The paper's lemma database has no instantiation of iteration constraints at positions (`TODO.md`: no instantiation at a witness); the two lemmas of `lsearch.dfy`, since merged into `teq.dfy`, abstract the constraint by an atom and instantiate the atom:

not_occurs_selects_def :

$$\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(i, 1, n) \rfloor\ \mathsf{check}\ k \mapsto a[k] \neq x)\ \iff\ \neg\mathsf{occurs}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(i, 1, n) \rfloor))$$

select_not_occurs_selects :

$$\neg\mathsf{occurs}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(i, 1, n) \rfloor))\ \wedge\ i \leq k < i + \mathsf{integer}(n)\ \implies\ (a[k] = x) \iff \mathsf{false}$$

The first matches the constraint with $i := 0$, $n := n'$ (the counter K is followed in the dependency position) and replaces it by the atom "x does not occur among the values of a at the positions $0, \ldots, n' - 1$". The second is a rule on the atom $a[k] = x$ of the last goal: its side conditions are the fact just produced, matched with $i := 0$, $n := n'$, and the bounds $0 \leq k < \mathsf{integer}(n')$, decided from $0 \leq k < i$ and $i = \mathsf{integer}(n')$. The hypothesis $a[k] = x$ becomes false and the goal closes.

This is the pattern of not_occurs_prefix_def and earlier_not_occurs_prefix of `teq.dfy` for reads $\mathsf{earlier}(k, xs)$ of a sequence, here for reads of an array: an iteration constraint becomes a named fact about the prefix visited, and a second lemma uses the fact at any position in the prefix. `TODO.md` notes that a side condition that is the iteration constraint itself would make the rule cycle (finding 19 of `NOTES.md`), which is why the abstraction by an atom is not merely convenient.

The file's header prescribes `-teq -split -simplify`, since without the split the disjunction of the two exits, $n \leq i \vee a[i] = x$, had to be split by the simplifier against the postcondition; with the current version the goal closes without `-split` as well.

2 Nested Sum: the Fragment Lemmas Alone

2.1 The program

```
method main(m: int, n: int) returns (c: int)
  requires 0 <= m && 0 <= n
  ensures c == m * n
{
  var i: int; var j: int;
  c := 0; i := 0;
  while i < m {
    j := 0;
    while j < n { j := j + 1; }
    c := c + j;
```

```
      i := i + 1;
    }
  }
```

The smallest nested loop whose result depends on the inner loop: c sums the final values of j, which are all n. There is nothing to summarize but the inner counter, so this example introduces the *fragment lemmas* of `fragments.dfy`, which every nested example uses (Section 4.3 states the first two in full), and shows how an iteration constraint reaches a lemma of `teq.dfy`.

2.2 The trace equation system

$$K = \mathsf{counter}(0,1,n') \qquad I = \mathsf{counter}(0,1,n') \qquad \mathsf{integer}(n') = m \qquad c = \mathsf{sum}(\hat{J}) \tag{4}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{N}, \hat{K}'\ \mathsf{check}\ k, i, n'', K' \mapsto K' = \mathsf{counter}(0,1,n'')) \tag{5}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{K}', \hat{J}'\ \mathsf{check}\ k, i, K', J \mapsto J = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \tag{6}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}, \hat{J}'\ \mathsf{check}\ k, i, j, J \mapsto j = \mathsf{current}(J)) \tag{7}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}'\ \mathsf{check}\ k, i, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j < n)) \tag{8}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}\ \mathsf{check}\ k, i, j \mapsto n \le j) \tag{9}$$

$$\implies c = m \cdot n$$

The trace of c is $\mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \hat{J}\ \mathsf{do}\ c, k, j \mapsto c + j)$, which the introduction rule of sums in `teq.dfy` folds and the acceleration rule $\mathsf{current}(\mathsf{sums}(x, V)) = x + \mathsf{sum}(V)$ turns into $c = \mathsf{sum}(\hat{J})$: the sum of the final values of the inner loop. The task is to show that $\hat{J}$ consists of n repeated m times.

2.3 The lemmas and their instantiation

Two lemmas of `fragments.dfy`, stated in full in Section 4.3, apply with $X := \lambda k.\, 0$ and $c := 1$: $\mathsf{invs_sums_counter}$ (the trace of an inner variable increased by c over the inner counter K', which is $\mathsf{counter}(0,1,n'')$ in the same iteration, is a counter from its initial value $X(k)$) turns (6) into $\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(0,1,n''))$, and $\mathsf{invs_current_counter}$ (the final value of an inner counter) turns (7) into $\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n''))$. Two more fragment lemmas of `fragments.dfy` resolve the test and the exit; both are stated for an inner counter from 0, since their side conditions must be in the normal form of the facts, where a general initial value $X(k) + \mathsf{integer}(n'')$ would not match $\mathsf{integer}(n'')$ (a higher-order summand matches a single summand):

$\mathsf{invs_invs_counter_upper_zero}$:

$$0 \le n \ \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(0,1,n''))$$
$$\implies$$
$$\mathsf{all}(\mathsf{for}\ ks, Js\ \mathsf{check}\ k, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ x \mapsto x < n))$$
$$\iff \mathsf{all}(\mathsf{for}\ ks, Ns\ \mathsf{check}\ k, n'' \mapsto \mathsf{integer}(n'') \le n)$$

$\mathsf{invs_exit_counter_zero}$:

$$\mathsf{all}(\mathsf{for}\ ks, Ns, js\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n''))$$
$$\wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns\ \mathsf{check}\ k, n'' \mapsto \mathsf{integer}(n'') \le n) \implies$$
$$\mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto n \le j) \iff \mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto j = n)$$

The first matches the test (8), with the counter fragment as its side condition (which binds $Ns := \hat{N}$); the inner constraint $j < n$ over $\lfloor \mathsf{counter}(0,1,n'') \rfloor$ is the lemma $\mathsf{invs_counter_upper}$ of `teq.dfy`, $n'' = 0 \vee (0 < n \wedge \mathsf{integer}(n'') - 1 < n)$, which the nonnegative bound simplifies to $\mathsf{integer}(n'') \le n$. The second matches the exit (9), finds the two facts, and states the final value: $j = n$ in each outer iteration. It is an equivalence under its side conditions, since $j = \mathsf{integer}(n'') \le n$.

The goal is now $\mathsf{sum}(\hat{J}) = m \cdot n$ with the fact $\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{J}\ \mathsf{check}\ k, j \mapsto j = n)$. In the body of that iteration constraint the simplifier reads j as $\mathsf{earlier}(\mathsf{natural}(k), \hat{J})$, so the fact is

$$\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n') \rfloor\ \mathsf{check}\ k \mapsto \mathsf{earlier}(\mathsf{natural}(k), \hat{J}) = n)$$

(the counter K is followed in the dependency position), which consts_prefix_def of `teq.dfy` abstracts by the atom $\mathsf{consts_prefix}(n', n, \hat{J})$, and sum_consts_prefix with that atom as its side condition rewrites $\mathsf{sum}(\hat{J})$ to $n \cdot \mathsf{integer}(n')$. (Until October 10 this was sum_invs_const, with the iteration constraint itself as the side condition, which cycles in general, see `TODO.md`.) The goal $\mathsf{integer}(n') \cdot n = m \cdot n$ with $\mathsf{integer}(n') = m$ is closed by a lemma mul_eq_cancel, $(a \cdot n = b \cdot n) \Leftrightarrow (n = 0 \vee a = b)$, since the simplifier substitutes the solution of $\mathsf{integer}(n') = m$ into polynomials but not into the product.

2.4 The idea

The fragments of an inner loop are a trace equation system over bound variables. The simplifier would resolve them if the bound variables of one fragment saw the equations of the others, which they do not (`NOTES.md`, the two problems of nested loops). The fragment lemmas are the four steps of that resolution as rewrite rules: the inner trace is a counter, its current value is its length, the test bounds the length, the exit fixes it. The same four lemmas serve every example with an inner counter from 0 (`nested_mult`, `row_sums`, `transpose`); with another initial value the test and the exit are instead kept as side conditions of a summary (Section 4.4).

3 Row Sums and Transpose: a Chain of Specific Lemmas

3.1 The program

```
method main(A: array<int, array<int, int>>, m: int, n: int) returns (s: array<int, int>)
  requires 0 <= m && 0 <= n
  ensures forall i: int :: 0 <= i < m ==>
    s[i] == sum(selects(A[i], butlast(counter(0, 1, natural(n)))))
{
  var i: int;
  var j: int;
  var t: int;
  i := 0;
  while i < m
  {
    t := 0;
    j := 0;
    while j < n
    {
      t := t + A[i][j];
      j := j + 1;
    }
    s := s[i := t];
    i := i + 1;
  }
}
```

The inner loop sums one row and the outer loop stores the sum at the position of its iteration; the transpose (`transpose.dfy`) is the same shape with an array-valued inner result, the stores of column k into B. Both are run with `-include teq:nested`, the module of the fragments of inner loops.

3.2 The trace equation system

With the module, the goal of the program alone is

$$K = I = \mathsf{counter}(0, 1, n') \qquad \mathsf{integer}(n') = m \tag{10}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{N}, \hat{K}'\ \mathsf{check}\ k, i, nn, K' \mapsto K' = \mathsf{counter}(0, 1, nn))$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k, nn, J \mapsto J = \mathsf{counter}(0, 1, nn)) \tag{11}$$

$$\hat{N} = \lfloor \mathsf{consts}(n', \mathsf{natural}(n)) \rfloor \qquad \mathsf{consts_prefix}(n', n, \hat{J}) \tag{12}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{T}', \hat{J}'\ \mathsf{check}\ k, i, T, J \mapsto T = \mathsf{sums}(0, \mathsf{selects}(A[k], \lfloor J \rfloor))) \tag{13}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{T}, \hat{T}'\ \mathsf{check}\ k, i, t, T \mapsto t = \mathsf{current}(T)) \tag{14}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}\ \mathsf{check}\ k, i, j \mapsto n \le j) \qquad \ldots$$

$$\implies 0 \le i < m \Rightarrow \mathsf{earlier}(\mathsf{natural}(i), \hat{T}) = \mathsf{sum}(\mathsf{selects}(A[i], \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor))$$

The module has resolved the inner counter (11), the exit (abstracted to consts_prefix) and the numbers of inner iterations (12): $\hat{N}$ is the constant sequence n. The sum of row k is the trace (13) over the inner counter, its result the fragment (14), and the postcondition reads the stored results at position i. What is left is to carry $\hat{N}$ into the counter, the counter into the sum, the sum into the result, and the results into the read: four substitutions between fragments over different dependency lists, which the simplifier does not make by itself.

3.3 The chain

The first version of this file stated the substitutions as general lemmas: a fragment about the inner counter J with a higher-order body $t = F(k, J)$ is stated about $\mathsf{counter}(0, 1, nn)$, and a read $\mathsf{earlier}(\mathsf{natural}(i), ts)$ is the body of the all over ts at i. Both have a general left-hand side, matching every fragment, respectively every read, together with an all side condition over the lists of the left-hand side; matching the condition builds the simplifier's context of the fact's combinator, where the reads of the dependencies are rewritten, on which the rule matches again, one level deeper, until the step budget is exhausted (finding 19 of `NOTES.md`, the cycle that `TODO.md` describes for sum_invs_const). The runs did not terminate, which the runners of the time reported as proofs, since a killed run prints no goal.

The version that verifies has one specific lemma per step, each with the fragment it depends on as a side condition over a fresh list variable, identified by an equation (Section 6); C_n abbreviates the counter $\lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor$:

$$\mathsf{invs_counter_consts}: \quad 0 \le n$$
$$\implies \mathsf{all}(\mathsf{for}\ ks, \lfloor \mathsf{consts}(m, \mathsf{natural}(n)) \rfloor, Js\ \mathsf{check}\ k, nn, J \mapsto J = \mathsf{counter}(0, 1, nn))$$
$$\iff \mathsf{all}(\mathsf{for}\ ks, Js\ \mathsf{check}\ k, J \mapsto \lfloor J \rfloor = C_n)$$

$$\mathsf{invs_sums_selects_counter}: \quad \mathsf{all}(\mathsf{for}\ ks, Js_2\ \mathsf{check}\ k, J \mapsto \lfloor J \rfloor = C_n) \ \wedge\ Js_2 = Js$$
$$\implies \mathsf{all}(\mathsf{for}\ ks, Ts, Js\ \mathsf{check}\ k, T, J \mapsto T = \mathsf{sums}(0, \mathsf{selects}(F(k), \lfloor J \rfloor)))$$
$$\iff \mathsf{all}(\mathsf{for}\ ks, Ts\ \mathsf{check}\ k, T \mapsto T = \mathsf{sums}(0, \mathsf{selects}(F(k), C_n)))$$

$$\mathsf{invs_current_def}: \quad \mathsf{all}(\mathsf{for}\ ks, Ts_2\ \mathsf{check}\ k, T \mapsto T = G(k)) \ \wedge\ Ts_2 = Ts$$
$$\implies \mathsf{all}(\mathsf{for}\ ks, ts, Ts\ \mathsf{check}\ k, t, T \mapsto t = \mathsf{current}(T))$$
$$\iff \mathsf{all}(\mathsf{for}\ ks, ts\ \mathsf{check}\ k, t \mapsto t = \mathsf{current}(G(k)))$$

$$\mathsf{invs_maps_sum}: \quad \mathsf{all}(\mathsf{for}\ ks, ts\ \mathsf{check}\ k, t \mapsto t = \mathsf{sum}(F(k)))$$
$$\iff ts = \mathsf{map}(\mathsf{for}\ ks\ \mathsf{yield}\ k \mapsto \mathsf{sum}(F(k)))$$

$$\mathsf{earlier_maps_counter}: \quad 0 \le i < \mathsf{integer}(m)$$
$$\implies \mathsf{earlier}(\mathsf{natural}(i), \mathsf{map}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, m) \rfloor\ \mathsf{yield}\ k \mapsto F(k))) = F(i)$$

The first matches (11) with the dependency $\lfloor \mathsf{consts}(m, \mathsf{natural}(n)) \rfloor$ as a pattern, which is matched against $\hat{N}$ through its equation (12) in the context (a variant with $\hat{N}$ as a variable and the equation as a side condition is in the file as well). The second rewrites (13) to a sum over the counter

$\lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor$, with $F := \lambda k.\ A[k]$. The third rewrites (14) with G the sum of row k, after which the database makes $\mathsf{current}(\mathsf{sums}(0, xs))$ into $\mathsf{sum}(xs)$; the fourth turns the pointwise fragment of the results into a map (specific to sums, so that the fragments $j = \mathsf{current}(J)$ of counters are left to the module); the fifth reads the map at i, with $m := n'$ and $0 \le i < \mathsf{integer}(n')$ decided from $0 \le i < m$ and (10). Every left-hand side is a fragment with a function symbol on the side of its bound variable or a read of a map, and every all condition is over a fresh list: nothing matches what the rewriting of a read in a combinator context produces, and the runs take under twenty seconds.

For the transpose the chain is the same with the inner stores: the trace of stores over $\lfloor J \rfloor$ is folded into $\mathsf{column}(h, A, k, \mathsf{natural}(n))$ once $J = \mathsf{counter}(0, 1, \mathsf{natural}(n))$ is known, the final values of the columns are the fragment $b = \mathsf{current}(\mathsf{column}(h, A, k, \mathsf{natural}(n)))$ over the previous value h, then current_results_steps (Section 2) makes the outer trace the trace of the columns, and current_transpose reads the transposed element.

3.4 The idea

An inner result stored at the position of the outer iteration is the pattern of the paper's nested loops, and with the module it needs no new concept, only discipline: every step is a lemma whose left-hand side is specific and whose side condition is a fragment over a fresh list, matched against the facts. The general substitution lemmas that one would write first are exactly the rules that the cycle mechanism of `TODO.md` forbids; the chain makes the same substitutions one fragment at a time.

4 Selection Sort: Fragments, a Summary, and an Induction

4.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var i: int; var j: int; var m: int; var t: int;
  b := a; i := 0;
  while i < n {
    m := i; j := i + 1;
    while j < n {
      if b[j] < b[m] { m := j; }
      j := j + 1;
    }
    t := b[i]; b[i] := b[m]; b[m] := t;
    i := i + 1;
  }
}
```

Arrays are total maps $\mathsf{array}\langle \mathsf{int}, \mathsf{int} \rangle$; $\mathsf{sorted}(a, lo, hi)$ states that $a[lo..hi)$ is nondecreasing, and $\mathsf{perm}(a, b)$ that a is a permutation of b as a total map, i.e. b composed with a bijection of the indices. Both are abstract predicates, declared without a definition; their properties are the lemmas below. Whole-array permutation needs no side conditions on the ranges of swapped positions: a swap at any two positions preserves it.

4.2 The trace equation system

Symbolic execution of the outer loop (Definition 4.4 of the paper) introduces the synthetic counter $K = \mathsf{counter}(0, 1, n')$ of n' iterations and one trace per modified variable. The inner loop is summarized during the execution of the body (Theorem 4.8): its own trace equation system is in terms of fresh variables, which become fresh *sequences* over the outer iterations, and it is recorded as the

iteration constraint of the outer loop. After `-simplify` distributes the constraint over its conjuncts, the goal is (variables renamed, the fragments of the inner loop grouped):

$$K = \mathsf{counter}(0,1,n') \qquad I = \mathsf{counter}(0,1,n') \qquad \mathsf{integer}(n') = n \qquad b = \mathsf{current}(B) \tag{15}$$

$$B = \mathsf{trace}(a,\ \mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{M}\ \mathsf{do}\ x,k,i,m \mapsto \mathsf{swap}(x,k,m)) \tag{16}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{M}, \hat{M}'\ \mathsf{check}\ k,i,m,M \mapsto m = \mathsf{current}(M)) \tag{17}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \lfloor B\rfloor, \hat{M}', \hat{J}'\ \mathsf{check}\ k,i,x,M,J \mapsto \\ M = \mathsf{trace}(k,\ \mathsf{for}\ \lfloor J\rfloor\ \mathsf{do}\ p,j \mapsto \mathsf{case}\ x[j] < x[p] \Rightarrow j\ \ \mathsf{case}\ x[p] \le x[j] \Rightarrow p)) \tag{18}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{N}, \hat{K}'\ \mathsf{check}\ k,i,n'',K' \mapsto K' = \mathsf{counter}(0,1,n'')) \tag{19}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{K}', \hat{J}'\ \mathsf{check}\ k,i,K',J \mapsto J = \mathsf{sums}(k+1, \mathsf{map}(\mathsf{for}\ \lfloor K'\rfloor\ \mathsf{yield}\ 1))) \tag{20}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{J}', \hat{J}\ \mathsf{check}\ k,i,J,j \mapsto j = \mathsf{current}(J)) \tag{21}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{J}'\ \mathsf{check}\ k,i,J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J\rfloor\ \mathsf{check}\ j \mapsto j < n)) \tag{22}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{J}\ \mathsf{check}\ k,i,j \mapsto n \le j) \tag{23}$$

$$\implies \mathsf{sorted}(b,0,n) \wedge \mathsf{perm}(b,a)$$

Equation (16) is the outer array trace: in iteration k (with $i = k$, since I and K are equal counters that the simplifier does not identify), the positions k and m are swapped, where m is the result of the inner loop, the entry of the sequence $\hat{M}$ at k. The fragments (17)–(23) describe that inner loop: its result m is the current value of a *selection trace* M (17), which starts at k and moves to j whenever $x[j] < x[p]$, over the inner counter J (18); J is the trace of j, increased by 1 from $k+1$ over the synthetic inner counter K' of n'' steps (19), (20), with the final value j (21); the test $j < n$ holds in every inner iteration (22) and fails at the exit (23). The generic fold of `teq.dfy` already turned the trace of j into $\mathsf{sums}(k+1, \mathsf{map}(\mathsf{for}\ \lfloor K'\rfloor\ \mathsf{yield}\ 1))$, since inside a fragment the equation $K' = \mathsf{counter}(0,1,n'')$ of the sibling fragment (19) is not followed (problem 1 of nested loops in `NOTES.md`).

4.3 Step 1: the inner counter (generic fragment lemmas)

The two lemmas of `fragments.dfy` relate fragments of one inner loop to each other. They are rewrite rules whose left-hand side is a fragment and whose side condition is a sibling fragment, matched against the facts of the goal, which binds the sequences that the left-hand side does not mention.

$$\begin{aligned}&\mathsf{invs_sums_counter}:\\ &\quad \mathsf{all}(\mathsf{for}\ ks, Ks, Ns\ \mathsf{check}\ k,K,n'' \mapsto K = \mathsf{counter}(0,1,n'')) \implies\\ &\mathsf{all}(\mathsf{for}\ ks, Ks, Js\ \mathsf{check}\ k,K,J \mapsto J = \mathsf{sums}(X(k), \mathsf{map}(\mathsf{for}\ \lfloor K\rfloor\ \mathsf{yield}\ c)))\\ &\quad \iff \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k,n'',J \mapsto J = \mathsf{counter}(X(k),c,n''))\end{aligned}$$

$$\begin{aligned}&\mathsf{invs_current_counter}:\\ &\quad \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k,n'',J \mapsto J = \mathsf{counter}(X(k),c,n'')) \implies\\ &\mathsf{all}(\mathsf{for}\ ks, Js, js\ \mathsf{check}\ k,J,j \mapsto j = \mathsf{current}(J))\\ &\quad \iff \mathsf{all}(\mathsf{for}\ ks, Ns, js\ \mathsf{check}\ k,n'',j \mapsto j = X(k) + \mathsf{integer}(n'') \cdot c)\end{aligned}$$

The initial value $X(k)$ is a higher-order pattern variable (Miller fragment): it matches 0, k, $k+1$, or the value of a further dependency, so one lemma covers all inner counters. The first lemma matches fragment (20) with $ks := \lfloor K\rfloor$, $Ks := \hat{K}'$, $Js := \hat{J}'$, $X := \lambda k.\, k+1$, $c := 1$; its side condition is found as fragment (19), which binds $Ns := \hat{N}$. The unused dependency $\lfloor I\rfloor$ of the fragment is selected by no pattern dependency and dropped. The result replaces (20):

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k,n'',J \mapsto J = \mathsf{counter}(k+1,1,n'')). \tag{24}$$

The second lemma matches (21) with $Js := \hat{J}'$, $js := \hat{J}$, finds (24) as its side condition, and yields

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}, \hat{J}\ \mathsf{check}\ k,n'',j \mapsto j = \mathsf{integer}(n'') + k + 1). \tag{25}$$

Conceptually, the fragment lemmas do by rewriting what the simplifier would do if the fragments of one inner loop shared their facts (the sharing was implemented and reverted, since it made rewriting cycle): they propagate the definition of a bound variable from one fragment into another. They are candidates for `teq.dfy`; they close the two minimal nested-loop problems `invs_counter_equation.dfy` and `invs_fragments.dfy`, and `nested_sum.dfy`, `nested_mult.dfy`, `distinct.dfy` and the sorting algorithms use them.

4.4 Step 2: the summary of the inner loop

The inner loop finds the position of the minimum of $x[k..n)$. We name that position by an abstract function $\mathsf{argmin}(x, lo, hi)$, declared without a definition, and state the summary as a rewrite rule whose side conditions are the fragments, in the forms they have after Step 1:

$$
\begin{aligned}
&\mathsf{current_selection_swaps}: \\
&\quad \mathsf{all}(\mathsf{for}\ ks, ms, Ms\ \mathsf{check}\ k, m, M \mapsto m = \mathsf{current}(M)) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, \lfloor \mathsf{trace}(a,\ \mathsf{for}\ ks, is, ms\ \mathsf{do}\ x, k, i, m \mapsto \mathsf{swap}(x, k, m))\rfloor, Ms, Js \\
&\qquad\quad \mathsf{check}\ k, x, M, J \mapsto \\
&\qquad\qquad M = \mathsf{trace}(k,\ \mathsf{for}\ \lfloor J\rfloor\ \mathsf{do}\ p, j \mapsto \mathsf{case}\ x[j] < x[p] \Rightarrow j\ \ \mathsf{case}\ x[p] \le x[j] \Rightarrow p)) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(k+1, 1, n'')) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns, js\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n'') + k + 1) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto n \le j) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Js_2\ \mathsf{check}\ k, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J\rfloor\ \mathsf{check}\ j \mapsto j < n_2)) \ \wedge\ Js_2 = Js \ \wedge\ n_2 = n \\
&\quad \Longrightarrow\ \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks, is, ms\ \mathsf{do}\ x, k, i, m \mapsto \mathsf{swap}(x, k, m))) \\
&\qquad\quad = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ x, k \mapsto \mathsf{swap}(x, k, \mathsf{argmin}(x, k, n))))
\end{aligned}
$$

The left-hand side is matched against $b = \mathsf{current}(B)$ with the definition (16) of B, which binds a, $ks := \lfloor K\rfloor$, $is := \lfloor I\rfloor$, $ms := \hat{M}$. The side conditions are then searched among the facts in order, each binding a new sequence: (17) binds $Ms := \hat{M}'$; (18) binds $Js := \hat{J}'$, where the dependency $\lfloor \mathsf{trace}(a,\ \mathsf{for}\ \ldots\ \mathsf{do}\ \rfloor)$ of the pattern is matched against $\lfloor B\rfloor$ through the definition of B; (24) binds $Ns := \hat{N}$; (25) binds $js := \hat{J}$; (23) binds n. The remaining condition is the inner test (22), which is matched by a duplicate Js_2 with a fresh bound n_2, identified afterwards by the equations $Js_2 = Js$ and $n_2 = n$: a side condition whose sequence variables are all bound is *decided* by the simplifier rather than matched against the facts, and the decision fails, since the facts keep $\lfloor I\rfloor$ as a dependency and rename the bound variables. The equations are closed and trivially decided.

Soundness of the lemma: given the side conditions, in iteration k the selection trace M scans the positions $k+1, \ldots, k+n''$ with $n'' + k + 1 \ge n$ (exit) and $k + 1 + n'' - 1 < n$ if $n'' > 0$ (test), hence exactly $k+1, \ldots, n-1$, so $\mathsf{current}(M)$ is a position of the minimum of $x[k..n)$, i.e. $\mathsf{argmin}(x, k, n)$ for any choice of argmin among the minima. The result is the trace of swaps of k with $\mathsf{argmin}(x, k, n)$, the *summary* of the whole algorithm as a trace of an abstract step. After the rewrite, $b = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor K\rfloor\ \mathsf{do}\ x, k \mapsto \mathsf{swap}(x, k, \mathsf{argmin}(x, k, n))))$.

Two workarounds are visible: the inner test is kept in its raw form (22) as a side condition instead of being resolved to a bound on n'', since the resolved form $k + 1 + \mathsf{integer}(n'') \le n$ is matched by `above_iterations_def` of `teq.dfy` (pattern $k < t$ with the dependency t synthesized as a map), which hides the fact; and the initial value $k+1$ and the final value $\mathsf{integer}(n'') + k + 1$ are written literally in the normal form of the simplifier, since a higher-order summand $X(k)$ matches a single summand only.

4.5 Step 3: sortedness by induction, permutation by a generic lemma

The postcondition $\mathsf{sorted}(b, 0, n)$ is now

$$\mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor K\rfloor\ \mathsf{do}\ x, k \mapsto \mathsf{swap}(x, k, \mathsf{argmin}(x, k, n)))), 0, n).$$

The invariant of selection sort after k iterations is that the prefix $x[0..k)$ is sorted and all of it is at most all of $x[k..n)$, written with a second abstract predicate $\mathsf{partitioned}(x, lo, mid, hi)$ (all of

$x[lo..mid)$ are at most all of $x[mid..hi)$). The induction is a lemma over a trace of arbitrary steps F:

$$\begin{aligned}&\mathsf{sorted_prefix_steps}:\\ &\quad n = \mathsf{integer}(m) \ \wedge\ \big(\forall x, k.\ 0 \le k < n \wedge \mathsf{sorted}(x,0,k) \wedge \mathsf{partitioned}(x,0,k,n)\\ &\qquad \Longrightarrow \mathsf{sorted}(F(x,k),0,k+1) \wedge \mathsf{partitioned}(F(x,k),0,k+1,n)\big)\\ &\quad \Longrightarrow\ \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,m)\rfloor\ \mathsf{do}\ x,k \mapsto F(x,k))),0,n) \iff \mathsf{true}\end{aligned}$$

The left-hand side matches with $F := \lambda x\, k.\ \mathsf{swap}(x,k,\mathsf{argmin}(x,k,n))$ and $m := n'$, through the definition $K = \mathsf{counter}(0,1,n')$ in the dependency position. The condition $n = \mathsf{integer}(m)$ is the fact $\mathsf{integer}(n') = n$ of (15); it fixes the number of steps to exactly n, since further steps would be unconstrained by the second condition (a lemma with $n \le \mathsf{integer}(m)$ would be unsound). The quantified condition is decided by the simplifier: F is beta-reduced, and the two conjuncts of the conclusion are rules on atoms,

$$\begin{aligned}&\mathsf{sorted_swap_argmin}:\\ &\quad k < n \wedge \mathsf{sorted}(x,0,k) \wedge \mathsf{partitioned}(x,0,k,n)\\ &\qquad \Longrightarrow \mathsf{sorted}(\mathsf{swap}(x,k,\mathsf{argmin}(x,k,n)),0,k+1) \iff \mathsf{true},\\ &\mathsf{partitioned_swap_argmin}:\\ &\quad k < n \wedge \mathsf{sorted}(x,0,k) \wedge \mathsf{partitioned}(x,0,k,n)\\ &\qquad \Longrightarrow \mathsf{partitioned}(\mathsf{swap}(x,k,\mathsf{argmin}(x,k,n)),0,k+1,n) \iff \mathsf{true},\end{aligned}$$

whose side conditions are the hypotheses of the quantified implication. The base case is inside the induction lemma: $\mathsf{sorted}(a,0,0)$ and $\mathsf{partitioned}(a,0,0,n)$ hold trivially (two further lemmas, sorted_empty and partitioned_empty, state them for completeness). This is the method of the paper's current_trace_bounded, an invariant scheme as a lemma with a quantified side condition: the lemma database proves the induction step, the shape of the invariant is the lemma's knowledge of selection sort, and no invariant inference takes place.

The postcondition $\mathsf{perm}(b,a)$ follows from a generic lemma on traces of swaps at positions that may depend on the state and the iteration (a higher-order pattern):

$$\begin{aligned}&\mathsf{perm_swaps_one}:\\ &\quad \mathsf{perm}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ x,k \mapsto \mathsf{swap}(x,P(x,k),Q(x,k)))),c)\\ &\quad \iff \mathsf{perm}(a,c)\end{aligned}$$

with $P := \lambda x\, k.\ k$ and $Q := \lambda x\, k.\ \mathsf{argmin}(x,k,n)$, and then perm_refl. The whole goal closes with `./Cuvee.sh -teq -simplify`; a wrong postcondition, e.g. $\mathsf{sorted}(b,0,n+1)$, leaves a goal.

4.6 What is generic and what is specific

Of the lemmas used, invs_sums_counter, invs_current_counter and perm_swaps_one are independent of the algorithm; sorted_prefix_steps is the induction for every algorithm that grows a sorted prefix (selection sort, and insertion sort in a variant with two dependencies); the summary current_selection_swaps and the two step lemmas are the algorithm's knowledge: what one inner loop computes, and why one step keeps the invariant. The abstract predicates and argmin are axiomatized by these lemmas only; a definition in Lean, as for `teq.dfy`, would make the step lemmas and the summary provable.

5 Bubble Sort: an Invariant Indexed by the Iteration

5.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
```

```
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var i: int; var j: int; var t: int;
  b := a; i := n;
  while 1 < i {
    j := 0;
    while j + 1 < i {
      if b[j + 1] < b[j] { t := b[j]; b[j] := b[j + 1]; b[j + 1] := t; }
      j := j + 1;
    }
    i := i - 1;
  }
}
```

5.2 The trace equation system

The outer loop modifies only b, and only through the inner loop, so its trace is the chain of the inner results, which the fold results names:

$$K = \mathsf{counter}(0, 1, n') \qquad I = \mathsf{counter}(n, -1, n') \qquad b = \mathsf{current}(B) \tag{26}$$

$$B = \mathsf{results}(a, \lfloor K \rfloor, \hat{B}) \quad \text{where} \quad \mathsf{results}(x_0, ks, bs) = \mathsf{trace}(x_0,\ \mathsf{for}\ ks, bs\ \mathsf{do}\ x, k, b \mapsto b)$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{B}, \hat{B}'\ \mathsf{check}\ k, i, b, B' \mapsto b = \mathsf{current}(B')) \tag{27}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor B \rfloor, \lfloor I \rfloor, \hat{B}', \hat{J}'\ \mathsf{check}\ k, x, i, B', J \mapsto B' = \mathsf{bubble_pass}(x, \lfloor J \rfloor)) \tag{28}$$

$$\text{where} \quad \mathsf{bubble_pass}(b, js) = \mathsf{trace}(b,\ \mathsf{for}\ js\ \mathsf{do}\ x, j \mapsto \mathsf{case}\ x[j+1] < x[j] \Rightarrow \mathsf{swap}(x, j, j+1)\ \ \mathsf{case}\ x[j] \le x[j+1] \Rightarrow x)$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}\ \mathsf{check}\ k, i, j \mapsto n \le j + k + 1) \tag{29}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}'\ \mathsf{check}\ k, i, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j + k + 1 < n)) \tag{30}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}, \hat{J}'\ \mathsf{check}\ k, i, j, J \mapsto j = \mathsf{current}(J)) \tag{31}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{K}', \hat{J}'\ \mathsf{check}\ k, i, K', J \mapsto J = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \tag{32}$$

$$\ldots \wedge\ n \le \mathsf{integer}(n') + 1\ \implies\ \mathsf{sorted}(b, 0, n) \wedge \mathsf{perm}(b, a)$$

The inner trace folds into $\mathsf{bubble_pass}(x, \lfloor J \rfloor)$, a pass over the positions of the inner counter J, by a `{:fold}` function of the file, like sums in `teq.dfy`. The test and the exit are stated with $i = n - k$ already substituted: the pass of iteration k runs over the positions $0, \ldots, n - k - 2$.

5.3 The summary

$$\begin{aligned}
&\mathsf{current_results_bubble}: \\
&\quad \mathsf{all}(\mathsf{for}\ ks, bs, Bs\ \mathsf{check}\ k, b, B' \mapsto b = \mathsf{current}(B')) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, \lfloor \mathsf{results}(a, ks, bs) \rfloor, Bs, Js\ \mathsf{check}\ k, x, B', J \mapsto B' = \mathsf{bubble_pass}(x, \lfloor J \rfloor)) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto len \le j + k + 1) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Js_2\ \mathsf{check}\ k, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j + k + 1 < len))\ \wedge\ Js_2 = Js \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, js_3, Js_3\ \mathsf{check}\ k, j, J \mapsto j = \mathsf{current}(J))\ \wedge\ js_3 = js\ \wedge\ Js_3 = Js \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ks, Js\ \mathsf{check}\ k, K', J \mapsto J = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \\
&\quad \implies \mathsf{current}(\mathsf{results}(a, ks, bs)) \\
&\qquad\quad = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ b, k \mapsto \mathsf{current}(\mathsf{bubble_pass}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(len - k - 1)) \rfloor))))
\end{aligned}$$

This is the summary in its original form (October 9), before the fragment lemmas: the inner counter is matched as sums, and the test and exit bind len. The left-hand side $\mathsf{current}(\mathsf{results}(a, ks, bs))$ is matched against $b = \mathsf{current}(B)$ through (26), binding $ks := \lfloor K \rfloor$ and $bs := \hat{B}$; (27) binds $Bs := \hat{B}'$, (28) binds $Js := \hat{J}'$, (29) binds $js := \hat{J}$ and $len := n$; (30), (31), (32) are matched through duplicates and the equations identify them, as in Section 4.4. The result is the trace of passes, the k-th over

$\lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n-k-1)) \rfloor$. Note that the arithmetic $n-k-1$ on the bound variable k is admissible on the right-hand side of a rule; on a left-hand side it would not be matched, which is why the original version of this file had a function $\mathsf{passlen}(len, k)$ for it, which the induction below makes unnecessary.

5.4 The induction with a clamped invariant

After k passes the suffix $b[n-k..n)$ is sorted and all of $b[0..n-k)$ are at most all of it. Stated with sorted and partitioned as in Section 4.5, the invariant makes sense only for $k \le n$, but the number of outer iterations n' is known only as $n \le n'+1$ together with the iteration constraint $1 < i$, a disjunction $n' = 0 \lor (2 < n \land n' < n)$ that the simplifier does not split, so the induction cannot fix the number of steps to $n-1$ (as Section 4.5 fixes it to n). Instead the invariant is a predicate indexed by the iteration, $\mathsf{bubbled}(b, k, n)$, which means the sorted suffix of length k once $k \le n$ and the whole range sorted once $k \ge n$; then every step keeps it, also the empty passes beyond $n-1$:

bubbled_pass :

$$p = len - k - 1 \land \mathsf{bubbled}(b, k, len)$$
$$\implies \mathsf{bubbled}(\mathsf{current}(\mathsf{bubble_pass}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(p)) \rfloor)), k+1, len) \iff \mathsf{true}$$

sorted_bubbled_steps :

$$len \le \mathsf{integer}(m) + 1$$
$$\land \ \big(\forall b, k.\ 0 \le k < \mathsf{integer}(m) \land \mathsf{bubbled}(b, k, len) \implies \mathsf{bubbled}(F(b,k), k+1, len)\big)$$
$$\implies \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, m) \rfloor\ \mathsf{do}\ b, k \mapsto F(b,k))), 0, len) \iff \mathsf{true}$$

The induction matches with $F := \lambda b\, k.\ \mathsf{current}(\mathsf{bubble_pass}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n-k-1)) \rfloor))$, $m := n'$, $len := n$. Its quantified condition is decided by bubbled_pass, whose length p is a variable with the equation $p = len-k-1$ as a side condition, since a pattern $len-k-1$ with two variables is not matched; the equation is closed after p is bound to $n-k-1$ by the left-hand side and decided by arithmetic. The lemma is sound for any number of steps m with $len \le m+1$: the invariant at $k = 0$ holds trivially, and $\mathsf{bubbled}(b, m, len)$ with $len - m \le 1$ is $\mathsf{sorted}(b, 0, len)$. For perm, perm_steps (the quantified condition $\forall b, k.\ \mathsf{perm}(F(b,k), b)$, decided by perm_bubble_pass) and perm_refl.

5.5 The idea

Two things differ from selection sort. The inner loop is summarized to a function of the previous array that is itself a trace, $\mathsf{current}(\mathsf{bubble_pass}(\ldots))$, so the outer trace is a trace of traces; the lemma database needs no knowledge of what a pass does beyond the two facts bubbled_pass and perm_bubble_pass. And the invariant is written as one predicate over the iteration, which absorbs the case distinction on the number of passes that the goal does not provide. The pattern "invariant as a predicate of (b, k), step lemma, generic induction" is the same as current_trace_bounded of `teq.dfy`, with the induction hypothesis as an abstract atom.

6 Cocktail Sort: Two Inner Loops and a Closed Side Condition

6.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var lo: int;
  var hi: int;
  var j: int;
  var p: int;
  var t: int;
```

```
  b := a;
  lo := 0;
  hi := n;
  while lo + 1 < hi
  {
    j := lo;
    while j + 1 < hi
    {
      if b[j + 1] < b[j] {
        t := b[j];
        b[j] := b[j + 1];
        b[j + 1] := t;
      }
      j := j + 1;
    }
    hi := hi - 1;
    j := hi - 1;
    p := hi - 2;
    while lo < j
    {
      if b[j] < b[p] {
        t := b[j];
        b[j] := b[p];
        b[p] := t;
      }
      j := j - 1;
      p := p - 1;
    }
    lo := lo + 1;
  }
}
```

Each outer iteration runs a forward bubble pass over $[lo, hi)$, which moves the maximum to $hi-1$, and a backward pass over $(lo, hi-1]$, which moves the minimum to lo; the backward pass uses two pointers j and $p = j-1$, like `insertion_sort2.dfy`, so that the positions of its swaps are counters.

6.2 The trace equation system

The outer loop modifies b only through the two inner loops, so its trace is again the chain of the results, $B = \mathsf{results}(a, \lfloor K \rfloor, \hat{B}'')$, where the intermediate results $\hat{B}'$ of the forward passes are the initial values of the backward passes. With $lo = k$ and $hi = n - k$ substituted,

$$K = Lo = \mathsf{counter}(0, 1, n') \qquad Hi = \mathsf{counter}(n, -1, n') \qquad b = \mathsf{current}(B) \qquad lo = \mathsf{integer}(n') \tag{33}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor \text{ check } k, lo, hi \mapsto 2k + 1 < n) \tag{34}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}', \hat{B}'_t \text{ check } k, lo, hi, c, C \mapsto c = \mathsf{current}(C))$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor B \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}'_t, \hat{J}'_t \text{ check } k, x, lo, hi, C, J \mapsto C = \mathsf{bubble_pass}(x, \lfloor J \rfloor)) \tag{35}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \hat{N}, \hat{J}'_t \text{ check } k, nn, J \mapsto J = \mathsf{counter}(k, 1, nn))$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{J}' \text{ check } k, lo, hi, j \mapsto n \le j + k + 1) \qquad \ldots \tag{36}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}'', \hat{B}''_t \text{ check } k, lo, hi, b, B' \mapsto b = \mathsf{current}(B'))$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}', \hat{B}''_t, \hat{J}''_t, \hat{P}''_t \text{ check } k, lo, hi, c, B', J, P \mapsto$$
$$B' = \mathsf{bubble_pass_back}(c, \lfloor J \rfloor, \lfloor P \rfloor)) \tag{37}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \hat{N}', \hat{J}''_t \text{ check } k, nn, J \mapsto J = \mathsf{counter}(n - k - 2, -1, nn))$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \hat{N}', \hat{P}''_t \text{ check } k, nn, P \mapsto P = \mathsf{counter}(n - k - 3, -1, nn)) \tag{38}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{J}'' \text{ check } k, lo, hi, j \mapsto j \le k)$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{J}'', \hat{J}''_t \text{ check } k, lo, hi, j, J \mapsto j = \mathsf{current}(J)) \qquad \ldots \tag{39}$$

$$\ldots \wedge\ n \le \mathsf{integer}(n') + lo + 1 \implies \mathsf{sorted}(b, 0, n) \wedge \mathsf{perm}(b, a)$$

with bubble_pass as in the bubble sort and bubble_pass_back(c, js, ps) the trace of conditional swaps swap(x, j, p) over the two counters. All facts of the outer iteration carry the counters Lo and Hi as dependencies, which no body uses, and the outer exit is $n \leq \mathsf{integer}(n') + lo + 1$ with $lo = \mathsf{integer}(n')$ a separate fact.

6.3 The summary and the induction

The summary current_results_shake takes the fragments of both inner loops as side conditions, twenty-one of them, in the manner of Section 4.4: the results and traces of each pass, the counters (whose initial values $n-k-2$, $n-k-3$ are higher-order $X(k)$, $Y(k)$, pinned by closed all conditions $\mathsf{all}(\mathsf{for}\ ks\ \mathsf{check}\ k \mapsto X(k) = len - k - 2)$, since a sum of a rule variable and a bound variable is not matched), the tests and the exits; the result is

$$\mathsf{current}(\mathsf{results}(a, ks, bs)) = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ b, k \mapsto \mathsf{shake}(b, k, len)))$$

with $\mathsf{shake}(b, k, len)$ abstract: the pass of iteration k. The induction is that of the bubble sort with the invariant $\mathsf{shaken}(b, k, len)$, both ends of length k sorted and the middle between them, the whole range once $2k + 1 \geq len$, and shaken_shake as the step.

The goal stayed open for a day although every piece was in place, and the diagnosis is the point of this section. Truncated copies of the summary with its first k side conditions showed that it fires with $k \leq 20$ and not with the 21st, $\mathsf{all}(\mathsf{for}\ ks, js_4, Js_3\ \mathsf{check}\ k, j, J \mapsto j = \mathsf{current}(J))$, the final values of the backward counter. All its variables were bound by earlier conditions, so the condition was closed and *decided* by the simplifier instead of matched against the fact (39), and the simplifier does not find a closed all among the facts (these carry the unused dependencies $\lfloor Lo \rfloor$, $\lfloor Hi \rfloor$, and their bound variables are renamed). The same condition for the forward pass had fresh variables js_5, Js_5 with identifying equations, the pattern of Section 4.4, and succeeded. With fresh js_6, Js_6 and $js_6 = js_4 \wedge Js_6 = Js_3$ the summary fires and perm closes.

Then $\mathsf{sorted}(b, 0, n)$ remained: the induction's bound $len \leq 2\,\mathsf{integer}(m) + 1$ is not a difference constraint, and the simplifier does not derive it from the two facts $n \leq \mathsf{integer}(n') + lo + 1$ and $lo = \mathsf{integer}(n')$. The lemma therefore states the bound in the goal's form,

$$\begin{aligned}&\mathsf{sorted_shaken_steps}: \quad l = \mathsf{integer}(m)\ \wedge\ len \leq \mathsf{integer}(m) + l + 1\\ &\quad \wedge\ \big(\forall b, k.\ 0 \leq k < \mathsf{integer}(m) \wedge \mathsf{shaken}(b, k, len) \Longrightarrow \mathsf{shaken}(F(b, k), k + 1, len)\big)\\ &\quad \Longrightarrow\ \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, m) \rfloor\ \mathsf{do}\ b, k \mapsto F(b, k))), 0, len)\ \Longleftrightarrow\ \mathsf{true}\end{aligned}$$

with l an open variable matched against the equation of lo, after which the bound is literally a fact. The counter $\lfloor \mathsf{counter}(0, 1, m) \rfloor$ as a dependency matches $\lfloor K \rfloor$ through $K = \mathsf{counter}(0, 1, n')$ in the context, as in the bubble sort. Two things that were suspected were not the cause: the extra dependencies $\lfloor Lo \rfloor$, $\lfloor Hi \rfloor$ of the facts (a diagnostic with two and with four dependencies matched both) and the name n of the rule variable, which the program shares (it was renamed to len anyway). The file in the tree now runs with the module of the fragments of inner loops (`-include teq:nested`), which resolves the inner counters to $\mathsf{counter}(k, 1, nn)$ and their final values to $j = \mathsf{integer}(nn) + k$, $\mathsf{integer}(nn) + j + k + 2 = len$ and $\mathsf{integer}(nn) + j + k + 3 = len$; the summary states these conditions in the forms the module produces, as the goal prints them, instead of $j = \mathsf{current}(J)$. The two insertion sorts were adapted the same way, so that every nested loop of the folder verifies with the module.

6.4 The idea

Nothing new is needed for two inner loops: each pass is summarized by its own fold, and the summary of the outer iteration takes both. The lesson is about side conditions: a condition whose variables are all bound is decided, not matched, so every all condition that should be read off a fact must bind a fresh sequence and be identified afterwards; and a closed arithmetic condition must be the fact as the goal has it, since the decision procedure combines difference constraints only.

7 Merge: Pinned Guards and Open Trace Variables

7.1 The program

```
method main(a: array<int, int>, m: int, b: array<int, int>, n: int) returns (c: array<int, int>)
  requires 0 <= m && 0 <= n
  requires sorted(a, 0, m) && sorted(b, 0, n)
  ensures sorted(c, 0, m + n)
  ensures forall x: int :: count(x, selects(c, butlast(counter(0, 1, natural(m + n)))))
    == count(x, selects(a, butlast(counter(0, 1, natural(m)))))
     + count(x, selects(b, butlast(counter(0, 1, natural(n)))))
{
  var i: int; var j: int; var k: int;
  i := 0; j := 0; k := 0;
  while k < m + n {
    if n <= j || (i < m && a[i] <= b[j]) { c := c[k := a[i]]; i := i + 1; }
    else { c := c[k := b[j]]; j := j + 1; }
    k := k + 1;
  }
}
```

The second postcondition states the permutation as a multiset: for each value, its number of occurrences among the first $m + n$ elements of c is the sum of its occurrences in a and b, in the vocabulary of `teq.dfy`.

7.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad c = \mathsf{current}(C) \qquad k = \mathsf{integer}(n') \qquad m + n \le k \tag{40}$$

$$C = \mathsf{trace}(c_0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \lfloor J \rfloor\ \mathsf{do}\ c, k, i, j \mapsto \mathsf{case}\ G \Rightarrow c[k := a[i]]\ \ \mathsf{case}\ \neg G \Rightarrow c[k := b[j]]) \tag{41}$$

$$I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor J \rfloor\ \mathsf{do}\ i, k, j \mapsto \mathsf{case}\ G \Rightarrow i + 1\ \ \mathsf{case}\ \neg G \Rightarrow i) \tag{42}$$

$$J = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ j, k, i \mapsto \mathsf{case}\ G \Rightarrow j\ \ \mathsf{case}\ \neg G \Rightarrow j + 1) \tag{43}$$

where G abbreviates the test $n \le j \vee (i < m \wedge a[i] \le b[j])$ of the program. The pointers are traces over each other, (42) and (43): I has $\lfloor J \rfloor$ among its dependencies and J has $\lfloor I \rfloor$. A single loop, no fragments; the whole difficulty is in matching this shape.

7.3 The lemmas

The merge is named by an abstract list $\mathsf{merged}(a, m, b, n)$, the values of the result on $0, \ldots, m+n-1$, with two facts: it is sorted, and its counts are those of both ranges. The lemma on the loop rewrites the values of the result:

$$\begin{aligned}
&\mathsf{selects_merge_loop}: \\
&\quad s = m + n\ \wedge\ m + n \le \mathsf{integer}(nn)\ \wedge\ 0 \le m \wedge 0 \le n \\
&\quad \wedge\ (\forall i, j.\ G(i, j) \Leftrightarrow (n \le j \vee (i < m \wedge a[i] \le b[j]))) \\
&\quad \wedge\ (\forall i, j.\ U(i, j) = a[i])\ \wedge\ (\forall i, j.\ V(i, j) = b[j]) \\
&\quad \wedge\ Iv = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, js\ \mathsf{do}\ i, k, j \mapsto \mathsf{case}\ G(i, j) \Rightarrow i + 1\ \ \mathsf{case}\ \neg G(i, j) \Rightarrow i) \\
&\quad \wedge\ is = \lfloor Iv \rfloor \\
&\quad \wedge\ Jv = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, is\ \mathsf{do}\ j, k, i \mapsto \mathsf{case}\ G(i, j) \Rightarrow j\ \ \mathsf{case}\ \neg G(i, j) \Rightarrow j + 1) \\
&\quad \wedge\ js = \lfloor Jv \rfloor \\
&\quad \Longrightarrow\ \mathsf{selects}(\mathsf{current}(\mathsf{trace}(c_0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, is, js\ \mathsf{do}\ c, k, i, j \mapsto \\
&\qquad\qquad \mathsf{case}\ G(i, j) \Rightarrow c[k := U(i, j)]\ \ \mathsf{case}\ \neg G(i, j) \Rightarrow c[k := V(i, j)])), \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(s)) \rfloor) \\
&\qquad = \mathsf{merged}(a, m, b, n)
\end{aligned}$$

and sorted_merge_loop is the same with the conclusion $\mathsf{sorted}(\mathsf{current}(\ldots), 0, s) \Leftrightarrow \mathsf{true}$ and the two sorted premises; count_merged gives

$$\begin{aligned}\mathsf{count}(x, \mathsf{merged}(a, m, b, n)) = \mathsf{count}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(m)) \rfloor)) \\ + \mathsf{count}(x, \mathsf{selects}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor)).\end{aligned}$$

Three workarounds of the matcher are built into this lemma, each isolated by experiments (`NOTES.md`, findings 14, 16 and the section on the pin):

1. **The pin.** A literal disjunctive guard $n \leq j \vee (\ldots)$ in a pattern is never matched. The guard is the higher-order variable $G(i, j)$, which matches any guard over the bound variables i, j, and the quantified side condition $\forall i, j.\ G(i, j) \Leftrightarrow (n \leq j \vee \ldots)$ pins it to the intended one; the simplifier decides the pin by beta reduction. The stored values $U(i, j) = a[i]$ and $V(i, j) = b[j]$ are pinned likewise. (A higher-order variable for an array read $a[i]$ alone would be instantiated by the array itself, finding 10, hence the pin instead of a literal $a[i]$.)
2. **The open trace variable.** The lemma must know that is and js are the pointer traces, which the pattern cannot say, since each is defined over the other. A closed side condition $is = \lfloor \mathsf{trace}(0,\ \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots) \rfloor$ is decided by the simplifier and fails (it does not expand $\lfloor I \rfloor$). Instead $Iv = \mathsf{trace}(0,\ \mathsf{for}\ \ldots, js\ \mathsf{do}\ \ldots)$ with an *open* trace variable Iv is matched against the hypothesis $I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor J \rfloor\ \mathsf{do}\ \ldots)$ (42), which binds $Iv := I$ (and the counter $\lfloor \mathsf{counter}(0, 1, nn) \rfloor$ matches $\lfloor K \rfloor$ through its definition); then $is = \lfloor Iv \rfloor$ is closed and trivially true.
3. **The whole atom as the left-hand side.** A variable argument of an uninterpreted function is not expanded by its definition when the simplifier rewrites below it: $\mathsf{sorted}(c, 0, m + n)$ with $c = \mathsf{current}(C)$ is not matched by a left-hand side $\mathsf{current}(\mathsf{trace}(\ldots,\ \mathsf{for}\ \ldots\ \mathsf{do}\ ))$, but $\mathsf{sorted}(\mathsf{current}(\mathsf{trace}(\ldots,\ \mathsf{for}\ \ldots\ \mathsf{do}\ )), 0, s)$ as the whole atom is, and $\mathsf{selects}(\mathsf{current}(\ldots), \ldots)$ with selects as the head, inside count and the quantifier. The length is a variable s with $s = m + n$ as a side condition, since a pattern $m + n$ of two variables is not matched.

The remaining condition $m + n \leq \mathsf{integer}(nn)$ is the exit $m + n \leq k$ with $k = \mathsf{integer}(n')$ of (40): after at least $m+n$ iterations the positions $0, \ldots, m+n-1$ are those of the merge, whatever further iterations store beyond them, which makes the lemma sound without fixing the number of iterations exactly (the goal knows it only as a disjunction).

7.4 The idea

The loop computes a function of its inputs that has a name in the specification's vocabulary, merged, and the lemma database relates the loop to that name and the name to the properties. The matching problems of mutually dependent traces are solved by two general devices, the pin and the open trace variable, which apply to every literal pattern that the matcher does not handle: in `insertion_sort.dfy` with one pointer, the positions j and $j - 1$ of the swaps are pinned in the same way, which closed a file that had been open since the start.

8 Gnome Sort: a Data-Dependent Number of Iterations

8.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var i: int;
  var t: int;
  b := a;
  i := 1;
  while i < n
  {
```

```
      if i == 0 || b[i - 1] <= b[i] {
        i := i + 1;
      } else {
        t := b[i];
        b[i] := b[i - 1];
        b[i - 1] := t;
        i := i - 1;
      }
    }
  }
```

The position moves up when the adjacent elements are in order and down with a swap otherwise. The number of iterations is not a counter and is unrelated to n: the exit $n \leq i$ bounds it from below only. The file was the folder's negative example, "open, as expected", until it was verified with two lemmas.

8.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad i = \mathsf{current}(I) \qquad b = \mathsf{current}(B) \qquad n \leq i \tag{44}$$

$$I = \mathsf{trace}(1,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor B \rfloor\ \mathsf{do}\ i, k, b \mapsto \mathsf{case}\ G \Rightarrow i + 1\ \ \mathsf{case}\ \neg G \Rightarrow i - 1) \tag{45}$$

$$B = \mathsf{trace}(a,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ b, k, i \mapsto \mathsf{case}\ G \Rightarrow b\ \ \mathsf{case}\ \neg G \Rightarrow b[i := b[i-1]][i-1 := b[i]]) \tag{46}$$

where G abbreviates $i = 0 \vee b[i-1] \leq b[i]$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, i \mapsto i < n) \tag{47}$$

As in the merge and the lower bound, two traces over each other with a data-dependent guard.

8.3 The lemmas

The invariant of gnome sort is $\mathsf{sorted}(b, 0, i)$: a step up extends the sorted prefix by the element that is in order with its predecessor, a swap down keeps the prefix below the position sorted, and at the exit $n \leq i$ the prefix covers the range. The lemma states this on the whole atom of the postcondition (Section 7), with the guard and the swap positions higher-order and pinned, since the literal $i - 1$ is not matched:

$$\begin{aligned}
\mathsf{sorted_gnome}: \quad & Bv = \mathsf{trace}(a,\ \mathsf{for}\ \lfloor Kv \rfloor, \lfloor Iv_2 \rfloor\ \mathsf{do}\ b, k, i \mapsto \mathsf{case}\ G(i,b) \Rightarrow b\ \ \mathsf{case}\ \neg G(i,b) \Rightarrow b[Q(i) := b[P(i)]][P(i) := b[Q \\
& \wedge\ Iv_2 = Iv \\
& \wedge\ Iv_2 = \mathsf{trace}(1,\ \mathsf{for}\ \lfloor Kv_2 \rfloor, \lfloor Bv_2 \rfloor\ \mathsf{do}\ i, k, b \mapsto \mathsf{case}\ G(i,b) \Rightarrow i + u\ \ \mathsf{case}\ \neg G(i,b) \Rightarrow i + d) \\
& \wedge\ Kv_2 = Kv \wedge Bv_2 = Bv \wedge u = 1 \wedge d = -1 \\
& \wedge\ (\forall i, b.\ G(i,b) \Leftrightarrow (i = 0 \vee b[i-1] \leq b[i]))\ \wedge\ (\forall i.\ P(i) = i - 1)\ \wedge\ (\forall i.\ Q(i) = i) \\
& \wedge\ x = \mathsf{current}(Iv)\ \wedge\ n \leq x \\
& \Longrightarrow\ \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor Kv \rfloor, \lfloor Iv \rfloor\ \mathsf{do}\ b, k, i \mapsto \mathsf{case}\ G(i,b) \Rightarrow b\ \ \mathsf{case}\ \neg G(i,b) \Rightarrow b[Q(i) := b[P(i)]][P(i) := b[Q(i)
\end{aligned}$$

The left-hand side matches $\mathsf{sorted}(b, 0, n)$ through $b = \mathsf{current}(B)$ and (46), binding $Kv := K$, $Iv := I$, G, $P := \lambda i.\ i - 1$, $Q := \lambda i.\ i$ by abstraction. The conditions follow the discipline of Section 10 and the lower bound: the trace of the array is tied first, open in Bv and Iv_2, matched against (46), and $Iv_2 = Iv$ is decided; the trace of the position is tied second, with fresh Kv_2, Bv_2 and the steps u, d as variables (a literal -1 in a pattern is not matched), matched against (45), and identified; the pins are decided by beta reduction; and the exit is read from the facts through $x = \mathsf{current}(Iv)$, matched against $i = \mathsf{current}(I)$, then $n \leq x$ decided. No condition mentions the left-hand side, so nothing re-enters. For the permutation, perm_gnome states that every step of such a trace is the identity or a swap, for any guard and positions, hence $\mathsf{perm}(\mathsf{current}(\ldots), c) \Leftrightarrow \mathsf{perm}(a, c)$, and perm_refl closes it. A wrong postcondition $\mathsf{sorted}(b, 0, n + 1)$ leaves the goal.

8.4 The idea

A data-dependent number of iterations is no obstacle when the property can be stated on the final values alone: the lemma does not describe how many steps were taken, only that the invariant

holds after any number of them and that the exit turns it into the postcondition. This is the same shape as the merge, where the pointers are traces over each other, and it shows that the "negative example" of the paper's loop shapes is negative for the characterization of the traces, not for the verification of the postcondition.

9 Counting Sort: Loops in Sequence and Atoms for Hypotheses

9.1 The program

```
method main(a: array<int, int>, n: int, k: int) returns (b: array<int, int>)
  requires 0 <= n && 0 < k
  requires keys_below(a, n, k)              // forall i :: 0 <= i < n ==> 0 <= a[i] < k
  ensures sorted(b, 0, n)
  ensures forall x: int :: count(x, selects(b, butlast(counter(0, 1, natural(n)))))
    == count(x, selects(a, butlast(counter(0, 1, natural(n)))))
{
  var c: array<int, int>; var i: int; var v: int; var s: int; var t: int;
  v := 0;
  while v < k { c := c[v := 0]; v := v + 1; }
  i := 0;
  while i < n { c := c[a[i] := c[a[i]] + 1]; i := i + 1; }
  s := 0; v := 0;
  while v < k { t := c[v]; c := c[v := s]; s := s + t; v := v + 1; }
  i := 0;
  while i < n { b := b[c[a[i]] := a[i]]; c := c[a[i] := c[a[i]] + 1]; i := i + 1; }
}
```

Four loops in sequence: the histogram is zeroed, filled, replaced by its prefix sums (the start position of each value), and the elements are scattered to their start positions, which are incremented.

9.2 The trace equation system

Each loop contributes a trace whose current value is the initial value of the next; the simplifier has already applied the introduction rules of `teq.dfy` (stores, consts, sums, selects) when the goal is printed (V, V_2, K_2 are counters of k' steps with $\mathsf{integer}(k') = k$, K, I, K_3, I_3 counters of n' steps with $\mathsf{integer}(n') = n$):

$$c_1 = \mathsf{current}(\mathsf{stores}(c_0, \lfloor V \rfloor, \lfloor \mathsf{consts}(k', 0) \rfloor)) \tag{48}$$

$$c_2 = \mathsf{current}(\mathsf{trace}(c_1,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ c, k, i \mapsto c[a[k] := c[a[k]] + 1])) \tag{49}$$

$$c_3 = \mathsf{current}(\mathsf{stores}(c_2, \lfloor V_2 \rfloor, \lfloor S \rfloor)) \qquad S = \mathsf{sums}(0, \mathsf{selects}(c_2, \lfloor K_2 \rfloor)) \tag{50}$$

$$b' = \mathsf{current}(\mathsf{stores}(b, \mathsf{map}(\mathsf{for}\ \lfloor K_3 \rfloor, \lfloor C \rfloor, \lfloor I_3 \rfloor\ \mathsf{yield}\ k, c, i \mapsto c[a[k]]), \mathsf{selects}(a, \lfloor I_3 \rfloor))) \tag{51}$$

$$C = \mathsf{trace}(c_3,\ \mathsf{for}\ \lfloor K_3 \rfloor, \lfloor I_3 \rfloor\ \mathsf{do}\ c, k, i \mapsto c[a[k] := c[a[k]] + 1]) \tag{52}$$

The scatter (51) is a stores whose keys are the positions read from the table C as it is incremented, and whose values are the elements of a.

9.3 The lemma chain

The histogram is characterized at each value (the generic lemma current_trace_add_at of `teq.dfy` is for a fixed key; here the key is $a[k]$):

$$\begin{aligned}&\mathsf{current_trace_histogram}: \\ &\quad \mathsf{current}(\mathsf{trace}(c_0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n) \rfloor, hs\ \mathsf{do}\ c, k, h \mapsto c[a[k] := c[a[k]] + 1]))[v] \\ &\qquad = c_0[v] + \mathsf{count}(v, \mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,n) \rfloor))\end{aligned}$$

It fires on the reads $c_2[v]$ inside $\mathsf{selects}(c_2, \lfloor K_2 \rfloor)$ of (50), with $c_1[v] = 0$ from (48) by current_stores of `teq.dfy`, so the prefix sums become

$$S = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K_2 \rfloor, \lfloor V_2 \rfloor\ \mathsf{yield}\ v, h \mapsto \mathsf{count}(v, \mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,n') \rfloor)))).$$

The table of start positions is then named by a lemma whose left-hand side is this whole term:

$$\begin{aligned}&\mathsf{rank_table_def}:\\&\quad\mathsf{current}(\mathsf{stores}(c_0,\lfloor\mathsf{counter}(0,1,kk)\rfloor,\lfloor\mathsf{sums}(0,\mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,kk)\rfloor,hs\ \mathsf{yield}\ v,h\mapsto\\&\qquad\mathsf{count}(v,\mathsf{selects}(a,\lfloor\mathsf{counter}(0,1,n)\rfloor)))))\rfloor)))=\mathsf{rank_table}(c_0,a,n,kk)\end{aligned}$$

which rewrites c_3 to $\mathsf{rank_table}(c_2,a,n',k')$, the number of elements of $a[0..n)$ below each value $v<k$ (and c_2 beyond). Naming the table is essential: the scatter lemma below needs the bound kk of the keys, and a side condition $\mathsf{ranks}(c_0,a,n,kk)$ with kk open would be matched by no fact, while $\mathsf{rank_table}(c_2,a,n,kk)$ in the pattern binds kk. Finally the scatter:

$$\begin{aligned}&\mathsf{sorted_scatter}:\\&\quad s=\mathsf{integer}(n)\ \wedge\ \mathsf{keys_below}(a,s,kb)\ \wedge\ kb=\mathsf{integer}(kk)\ \wedge\ Iv=\mathsf{counter}(0,1,n)\ \wedge\ hs_3=\lfloor Iv\rfloor\\&\quad\implies\mathsf{sorted}(\mathsf{current}(\mathsf{stores}(b_0,\mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor,\lfloor T\rfloor,hs_2\ \mathsf{yield}\ k,c,h\mapsto c[a[k]]),\\&\qquad\qquad\mathsf{selects}(a,hs_3))),0,s)\iff\mathsf{true}\\&\qquad\text{where } T=\mathsf{trace}(\mathsf{rank_table}(c_2,a,n,kk),\\&\qquad\qquad\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor,hs\ \mathsf{do}\ c,k,h\mapsto c[a[k]:=c[a[k]]+1])\end{aligned}$$

and count_scatter with the conclusion

$$\mathsf{count}(x,\mathsf{selects}(\mathsf{current}(\ldots),\lfloor\mathsf{counter}(0,1,n)\rfloor))=\mathsf{count}(x,\mathsf{selects}(a,\lfloor\mathsf{counter}(0,1,n)\rfloor)).$$

The left-hand side binds $n:=n'$, $kk:=k'$, $hs_3:=\lfloor I_3\rfloor$ and the rest; the counter of the values' selects is tied by the open trace variable Iv (Section 7.3), since $\lfloor I_3\rfloor$ and $\lfloor\mathsf{counter}(0,1,n')\rfloor$ are not identified by a closed equation. The precondition on the keys is the atom keys_below of the specification with an open bound kb, matched against the fact and then identified with $\mathsf{integer}(kk)$: a quantified hypothesis $\forall i.\ 0\le i<n\Rightarrow 0\le a[i]<k$ is matched by no side condition, open or closed, so the specification uses the atom (as `count_occurrences.dfy` uses the vocabulary of `teq.dfy` in its postcondition).

Soundness of sorted_scatter: with the start positions correct for all keys, element $a[k]$ is stored at (number of elements below $a[k]$) + (number of equal elements before position k), which is a bijection of $0,\ldots,n-1$ that is monotone in the value, so the result is sorted on $0,\ldots,n-1$ and has the elements of a. Both are asserted, not proved; the lemma is the algorithm's correctness argument in one statement, over the exact shape of the two last loops.

9.4 The idea

Loops in sequence compose through the current values: each loop's trace has the previous result as its initial value, and the acceleration lemmas of `teq.dfy` (stores read at a position, sums, consts) push the characterization of one loop into the next without any lemma specific to counting sort, up to the table of start positions. The algorithm-specific knowledge is concentrated in naming the table and in the scatter lemma, whose left-hand side is the composition of the last two loops. What could not be expressed is a quantified precondition as a side condition; the specification's vocabulary must provide an atom for it.

10 First Duplicate: a Search with an Early Exit

10.1 The program

```
method main(a: array<int, int>, n: int) returns (r: bool)
  requires 0 <= n
  ensures r ==> !distincts(selects(a, butlast(counter(0, 1, natural(n)))))
  ensures !r ==> distincts(selects(a, butlast(counter(0, 1, natural(n)))))
{
  var i: int; var j: int;
  r := false; i := 0;
```

```
    while i < n && !r {
      j := i + 1;
      while j < n && !r {
        if a[i] == a[j] { r := true; }
        j := j + 1;
      }
      i := i + 1;
    }
  }
```

The nested loop of `distinct.dfy` (which is verified) with both loops stopped by the flag: the search for a duplicate ends at the first one. The postcondition uses distincts of the lemma database, and is stated as two implications rather than the equivalence $r \Leftrightarrow \neg\mathsf{distincts}(\ldots)$, for a reason explained below.

10.2 The trace equation system

The outer loop's constraint includes the flag, $\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor R\rfloor, \lfloor I\rfloor\ \mathsf{check}\ k, r, i \mapsto \neg r)$, so r is false at the head of every outer iteration, and the simplifier uses this inside the inner trace:

$$K = I = \mathsf{counter}(0, 1, n') \qquad r = \mathsf{current}(R) \qquad R = \mathsf{results}(\mathsf{false}, \lfloor K\rfloor, \hat{R}) \qquad i = \mathsf{integer}(n') \tag{53}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor R\rfloor, \lfloor I\rfloor\ \mathsf{check}\ k, r, i \mapsto \neg r) \tag{54}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}, \hat{R}'\ \mathsf{check}\ k, i, r, R' \mapsto r = \mathsf{current}(R')) \tag{55}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}', \hat{J}'\ \mathsf{check}\ k, i, R', J \mapsto R' = \mathsf{trace}(\mathsf{false},\ \mathsf{for}\ \lfloor J\rfloor\ \mathsf{do}\ r, j \mapsto \mathsf{case}\ a[k] = a[j] \Rightarrow \mathsf{true}\ \ \mathsf{case}\ a[k] \neq a[j] \Rightarrow \mathsf{false})) \tag{56}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(k+1, 1, n'')) \tag{57}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}, \hat{J}\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n'') + k + 1) \tag{58}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{J}'\ \mathsf{check}\ k, i, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J\rfloor\ \mathsf{check}\ j \mapsto j < n)) \tag{59}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}'\ \mathsf{check}\ k, i, R' \mapsto \mathsf{all}(\mathsf{for}\ \lfloor R'\rfloor\ \mathsf{check}\ r \mapsto \neg r)) \tag{60}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}, \hat{J}\ \mathsf{check}\ k, i, r, j \mapsto \neg(j < n \wedge \neg r)) \tag{61}$$

$$\neg(i < n \wedge \neg r) \implies r \Leftrightarrow \neg\mathsf{distincts}(\mathsf{selects}(a, \lfloor\mathsf{counter}(0, 1, \mathsf{natural}(n))\rfloor))$$

(after the fragment lemmas of Section 4.3, which gave (57) and (58)). The inner update (56) does not mention the previous value of the flag, since the inner constraint $\neg r$ (60) replaced it by false: the inner trace is a sequence of comparisons with $a[k]$. The inner exit (61) is the negation of the test $j < n \wedge \neg r$: either all later positions were visited, or the flag was set. The outer exit $\neg(i < n \wedge \neg r)$ is a disjunction, $n \leq i \vee r$, with one goal per postcondition.

10.3 The lemmas

The summary of the inner loop in the style of Section 4.4 is a rewrite of the fragment (55) into a fact about the results: with (56)–(61) as side conditions (duplicated and identified by equations where closed),

$$\mathsf{all}(\mathsf{for}\ ks, rs, Rs\ \mathsf{check}\ k, r, R' \mapsto r = \mathsf{current}(R')) \iff \mathsf{all}(\mathsf{for}\ ks, rs\ \mathsf{check}\ k, r \mapsto r = \neg\mathsf{nodup}(a, k, n))$$

where $\mathsf{nodup}(a, k, n)$ is the abstract "$a[k]$ does not occur among $a[k+1..n)$". This is sound in both exit cases of the inner loop: if the flag was set, a later duplicate was found; if j reached n, all later positions were compared. The outer trace is then $\mathsf{results}(\mathsf{false}, \lfloor K\rfloor, \hat{R})$ with $\hat{R}$ characterized pointwise, and the simplifier itself turns it into $\mathsf{results}(\mathsf{false}, \lfloor K\rfloor, \mathsf{map}(\mathsf{for}\ \lfloor K\rfloor\ \mathsf{yield}\ k \mapsto \neg\mathsf{nodup}(a, k, n)))$.

The outer loop is summarized on the atom $\mathsf{distincts}(\ldots)$ of the postcondition, by one rule per case of the exit. If the flag is set, it is the result of the last iteration, which found a duplicate:

$$\begin{aligned}&\mathsf{distincts_flag_set}: \quad Kv = \mathsf{counter}(0, 1, m)\\ &\quad \wedge\ flag = \mathsf{current}(\mathsf{results}(\mathsf{false}, \lfloor Kv\rfloor, \mathsf{map}(\mathsf{for}\ \lfloor Kv\rfloor\ \mathsf{yield}\ k \mapsto \neg\mathsf{nodup}(a, k, n))))\\ &\quad \wedge\ flag\ \wedge\ \mathsf{integer}(m) \leq n \implies \mathsf{distincts}(\mathsf{selects}(a, \lfloor\mathsf{counter}(0, 1, \mathsf{natural}(n))\rfloor)) \iff \mathsf{false}\end{aligned}$$

If all iterations ran, $n \leq \mathsf{integer}(m)$, and the flag was reset at the head of each, (54), then no element before the last has a later duplicate (the flag at the head of iteration k is the result of iteration $k-1$) and the last has none:

$$\begin{aligned}\mathsf{distincts_flags_reset}: \quad & Kv = \mathsf{counter}(0,1,m) \ \wedge \ Rv = \mathsf{results}(\mathsf{false}, \lfloor Kv \rfloor, rs) \\ & \wedge \ \mathsf{all}(\mathsf{for} \ \lfloor Kv \rfloor, hs \ \mathsf{check} \ k, h \mapsto \neg h) \ \wedge \ hs = \lfloor Rv \rfloor \\ & \wedge \ \mathsf{all}(\mathsf{for} \ \lfloor Kv \rfloor, rs_2 \ \mathsf{check} \ k, h \mapsto h = \neg\mathsf{nodup}(a,k,n)) \ \wedge \ rs_2 = rs \ \wedge \ n \leq \mathsf{integer}(m) \\ & \implies \mathsf{distincts}(\mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,\mathsf{natural}(n)) \rfloor)) \iff \mathsf{true}\end{aligned}$$

Three details made these fire. The outer counter and the flag trace are the variables K and R of the goal with their equations (53) as hypotheses, so a rule with $\lfloor \mathsf{counter}(0,1,m) \rfloor$ as an *argument* never matches; the open trace variables Kv, Rv are matched against the hypotheses (Section 7). The fact of the flags at the heads (54) carries the counter $\lfloor I \rfloor$ as an unused dependency, which the simplifier drops from a closed instance of the condition, after which it is not found among the facts; hence the open list hs, identified with $\lfloor Rv \rfloor$ afterwards. And the exit $n \leq i \vee r$ is needed as a closed condition: with the postcondition as an equivalence, the goal's conclusion is $i < n \wedge \neg r \ \vee \ r \Leftrightarrow \neg\mathsf{distincts}(\ldots)$, which `-split` does not split, and in the context of the second disjunct neither literal of the negated first is available, so the condition is not decided; probes (rules with the real left-hand side and the first k conditions) located it as the only failing one. As two implications, each goal has its literal of the exit, r respectively $n \leq i$ with $i = \mathsf{integer}(n')$, in the context of the atom.

An earlier set of lemmas closed both goals and also the negated postcondition: the rule for the first step had the summary fragment as an open side condition, and a rule $\mathsf{nodup}(a,k,n) \Leftrightarrow \mathsf{true}$ under the hypothesis $\mathsf{distincts}(\ldots)$ for the second. An all side condition is matched in the simplifier's context of the fact's combinator, where rules fire: under the hypothesis distincts, the rule rewrote the condition to (54), a fact, so distincts was derived from itself (finding 13 of `NOTES.md`). The rules above take the atom distincts as left-hand side and never rewrite nodup.

10.4 The idea

An early exit is a data-dependent number of iterations with a disjunctive exit; the summary of the inner loop must be stated so that it holds in both cases, which the abstract atom nodup affords, since it describes the result rather than the number of iterations; the outer loop is summarized on the atom of the postcondition with one rule per case of its exit, each with the case as a closed condition. The soundness lesson stands: a side condition that other rules can simplify under the conclusion to be proved is a circular justification, and the safe form of side conditions is an atom that is matched literally (as consts_prefix_def of the database does for a constraint, and as the summary itself does for the inner loop).

11 Binary Search: the Lower Bound (open)

11.1 The program

```
method main(a: array<int, int>, n: int, x: int) returns (lo: int)
  requires 0 <= n
  requires sorted(a, 0, n)
  ensures 0 <= lo <= n
  ensures forall k: int :: 0 <= k < lo ==> a[k] < x
  ensures forall k: int :: lo <= k < n ==> x <= a[k]
{
  var hi: int; var mid: int;
  lo := 0; hi := n;
  while lo < hi {
    mid := (lo + hi) / 2;
    if a[mid] < x { lo := mid + 1; } else { hi := mid; }
  }
}
```

The first position of $a[0..n)$ whose element is at least x, for a sorted array. The example `bsearch.dfy` of `examples/dafny/teq` verifies a binary search with the stronger precondition of partition points for all indices; here the precondition is $\mathsf{sorted}(a, 0, n)$, so the comparisons $a[m] < x$ are meaningful for $0 \le m < n$ only.

11.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad lo = \mathsf{current}(L) \qquad hi = \mathsf{current}(H) \qquad hi \le lo \tag{62}$$

$$L = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor H \rfloor\ \mathsf{do}\ lo, k, hi \mapsto \mathsf{case}\ G \Rightarrow (hi + lo)/2 + 1\ \ \mathsf{case}\ \neg G \Rightarrow lo) \tag{63}$$

$$H = \mathsf{trace}(n,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor L \rfloor\ \mathsf{do}\ hi, k, lo \mapsto \mathsf{case}\ G \Rightarrow hi\ \ \mathsf{case}\ \neg G \Rightarrow (hi + lo)/2) \tag{64}$$

where G abbreviates $a[(hi + lo)/2] < x$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor L \rfloor, \lfloor H \rfloor\ \mathsf{check}\ k, lo, hi \mapsto lo < hi) \tag{65}$$

As in the merge, the two variables are traces over each other, with a data-dependent guard that reads the array at a position computed from both.

11.3 The attempt

The intended lemmas name the result by an abstract function $\mathsf{lb}(a, n, x)$ with the facts $0 \le \mathsf{lb}(a, n, x) \le n$, $(a[k] < x) \Leftrightarrow \mathsf{true}$ for $0 \le k < \mathsf{lb}(a, n, x)$, and $(x \le a[k]) \Leftrightarrow \mathsf{true}$ for $\mathsf{lb}(a, n, x) \le k < n$ (all under $\mathsf{sorted}(a, 0, n)$), which close the postcondition once $lo = \mathsf{lb}(a, n, x)$ is known. The summary of the loop in the style of the merge (Section 7): the guard is the higher-order $G(lo, hi)$ pinned by $\forall lo, hi.\ G(lo, hi) \Leftrightarrow a[(hi + lo)/2] < x$, the trace of hi is tied by an open trace variable $Hv = \mathsf{trace}(n,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, \lfloor Lv \rfloor\ \mathsf{do}\ \ldots)$ with $hs = \lfloor Hv \rfloor$, and the left-hand side is $\mathsf{current}(\mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, hs\ \mathsf{do}\ lo, k, hi \mapsto \mathsf{case}\ G(lo, hi) \Rightarrow (hi + lo)/2 + 1\ \ \mathsf{case}\ \neg G(lo, hi) \Rightarrow lo))$, rewritten to $\mathsf{lb}(a, n, x)$. What the lemma must also know is that the loop has terminated, the exit $hi \le lo$ of (62): without it the current value of L is not the lower bound.

Stating the exit as a side condition $\mathsf{current}(Hv) \le \mathsf{current}(Lv)$ makes the run exceed the time limit: to decide the condition, the simplifier rewrites $\mathsf{current}(Lv)$, which is the left-hand side of the very lemma, whose conditions decide it again (finding 20 of `NOTES.md`). Stating it as a comparison $h \le l$ of two open variables, matched against the hypothesis $hi \le lo$, makes the lemma fire, and the four facts about lb then close the three postconditions: the goal is closed, and a wrong postcondition leaves a goal. But h and l are not tied to the traces, so the rule is not a sound lemma (in this goal its conclusion holds whichever comparison the condition matched, since the exit does hold in the final state). Every sound tie fails for a reason of the engine, located with probes: the instance of a side condition is simplified before it is decided or matched, so $l = \mathsf{current}(Lv)$ with Lv bound re-enters the lemma like the first form, whether closed or open. With Lv open, $l = \mathsf{current}(Lv)$ binds both from the hypothesis $lo = \mathsf{current}(L)$ without re-entrance; but then the left-hand side, matched against $\mathsf{current}(L)$ through the definition of the trace variable L, turns out to have left its dependency variables nn and hs unbound (a probe fired with "variables of a rule not instantiated"), and the trace condition that would bind them, $Lv = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, hs\ \mathsf{do}\ \ldots)$, no longer matches once Lv is bound, since its instance is simplified with L replaced by its definition.

11.4 The idea, and what is missing

Binary search is the paper's acknowledged weakness: the loop computes a bound by narrowing, and only a partial description of the traces (monotone lo, monotone hi, $lo \le \mathsf{lb} \le hi$ throughout) is needed, not their values. `bsearch.dfy` handles it by abstracting the comparisons with the array into comparisons of indices with partition points (lemmas $(a[m] < x) \Leftrightarrow m < p$ without side conditions), by the named monotone traces incs and decs whose bounds are their facts, and by a cut after the loop. With sorted instead of partition points, the comparison lemma needs $0 \le m < n$, which in the context of the updates follows from the invariant $0 \le lo \le hi \le n$, the thing to be established; the summary lemma of this section bypasses the invariant by stating the result directly, and founders on

tying the exit to the traces soundly. What is missing is in the engine: a rule should not be applied while the instances of its own side conditions are simplified, and a left-hand side matched through a trace variable should bind all its variables; with either, the sound form of the lemma closes the goal.

12 The MonoCera Tests: Array Programs in C

The following sections re-trace the proofs of some of the tests of the MonoCera evaluation (Amilon, Esen, Gurov, Lidström, Rümmer, *Automatic Program Instrumentation for Automatic Verification*, CAV 2023): 125 C programs derived from SV-COMP, each with a loop that fills an array and a loop that checks an assertion over it, one test per aggregation operator that the assertion admits (∀, max, min, ∑). They are in `benchmarks/monocera/polished_tests/cuvee`, converted as for the other tools of that evaluation (macros substituted, `malloc` replaced by arrays, the SV-COMP harness kept), and run with `-unbounded -teq -simplify` as the benchmark definition does (mathematical integers, as the paper runs all tools); `benchmarks/monocera/summary.md` has the comparison (Table 2) and `lemmas.tex` there the case studies from which the lemmas came. Unlike the algorithms of the previous sections, these programs carry no specification and no lemmas of their own: the assertion is the postcondition, and everything the proof needs is in the database. The traces below are those of `-rules` and `-debug rewrite` with Cuvée 0.41.6 on October 12, 2026; the steps on the harness (`truth`, `number`, the bounds of `sizeof`) and the repeated reads of counters are left out.

Two things differ from the Dafny examples. The C frontend translates a `for` loop with a body of conditionals into a trace with one case per path, also where the paths agree (the counter i of the first loop of every test has two cases $i + 1$), and the simplifier merges such cases first (trace_merge_cases_ne). And the symbolic execution of `main` branches on the harness, `if (N <= 0) return 1` and `assume_abort_if_not(N <= 2147483647 / sizeof(int))`, so the goal has a branch per path of the harness, of which one carries the loops; the others are trivial.

13 Alternating Values: a Sum Bounded Through Three Lemmas

13.1 The program

The test `sum/brs2`: the array alternates between 2 and 0, the sum is kept in the cell 0 of a one-element array, and the assertion is an upper bound.

```
for (i = 0; i < N; i++) { if (i % 2 == 0) a[i] = 2; else a[i] = 0; }
for (i = 0; i < N; i++) { if (i == 0) sum[0] = 0; else sum[0] = sum[0] + a[i]; }
__VERIFIER_assert(sum[0] <= 2 * N);
```

13.2 The trace equation system

Two loops in sequence; the second one's trace of `sum` has the final array a' of the first in its update. With $N > 0$ from the harness,

$$K = \mathsf{counter}(0, 1, n) \qquad a' = \mathsf{current}(A) \qquad i' = \mathsf{current}(I) \tag{66}$$

$$A = \mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ a, k, i \mapsto \mathsf{case}\ i \bmod 2 = 0 \Rightarrow a[i := 2]\ \ \mathsf{case}\ i \bmod 2 \neq 0 \Rightarrow a[i := 0]) \tag{67}$$

$$I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ i, k \mapsto \mathsf{case}\ i \bmod 2 = 0 \Rightarrow i + 1\ \ \mathsf{case}\ i \bmod 2 \neq 0 \Rightarrow i + 1) \tag{68}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor A \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, a, i \mapsto i < N) \qquad \neg(i' < N) \tag{69}$$

$$K' = \mathsf{counter}(0, 1, n') \qquad s' = \mathsf{current}(S) \qquad i'' = \mathsf{current}(I')$$

$$S = \mathsf{trace}(s_0,\ \mathsf{for}\ \lfloor K' \rfloor, \lfloor I' \rfloor\ \mathsf{do}\ s, k, i \mapsto \mathsf{case}\ i = 0 \Rightarrow s[0 := 0]\ \ \mathsf{case}\ i \neq 0 \Rightarrow s[0 := s[0] + a'[i]]) \tag{70}$$

$$I' = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K' \rfloor\ \mathsf{do}\ i, k \mapsto i + 1) \qquad \mathsf{all}(\mathsf{for}\ \lfloor K' \rfloor, \lfloor S \rfloor, \lfloor I' \rfloor\ \mathsf{check}\ k, s, i \mapsto i < N) \qquad \neg(i'' < N)$$

$$\Longrightarrow\ s'[0] \leq 2N$$

(the `rem` of C is the remainder of the prelude, which the simplifier turns into mod under $0 \leq i$). The accumulator is not a scalar but a cell of an array, so (70) is a trace of stores at the constant key 0, and the array it reads is the current value of another trace of stores.

13.3 The lemmas

Three lemmas of the database cooperate, in the order in which the goal needs them.

stores_index_cases_def :

$$\mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor, \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{do}\ a,k,i \mapsto \mathsf{case}\ P(k) \Rightarrow a[k := c]\ \ \mathsf{case}\ \neg P(k) \Rightarrow a[k := d])$$
$$= \mathsf{stores}(a_0, \lfloor\mathsf{counter}(0,1,n)\rfloor, \mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ P(k)\ \mathsf{then}\ c\ \mathsf{else}\ d))$$

select_current_trace_reset_add :

$$\mathsf{current}(\mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor, \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{do}\ a,k,i \mapsto \mathsf{case}\ k = 0 \Rightarrow a[j := c]\ \ \mathsf{case}\ 0 < k \Rightarrow a[j := a[j] + G$$
$$= \mathsf{if}\ n = 0\ \mathsf{then}\ a_0[j]\ \mathsf{else}\ c + \mathsf{sum}(\mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ 0 < k\ \mathsf{then}\ G(k)\ \mathsf{else}\ 0))$$

sum_cases_below : $\quad 0 \le e\ \wedge\ \mathsf{integer}(n) \cdot c \le e\ \wedge\ \mathsf{integer}(n) \cdot d \le e$

$$\Longrightarrow\ \mathsf{sum}(\mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ 0 < k\ \mathsf{then}\ (\mathsf{if}\ P(k)\ \mathsf{then}\ c\ \mathsf{else}\ d)\ \mathsf{else}\ 0)) \le e\ \Longleftrightarrow\ \mathsf{true}$$

The first turns a loop that stores at each position a constant chosen by cases on the *index* into stores of a map; the second reads a sum kept in one cell j of an array as the scalar sum, with the right-hand side of current_trace_reset_add for a scalar accumulator, so that the database continues from there; the third bounds a sum of n terms that are each 0, c or d, with its left-hand side a comparison whose constants bind the bound e (the general sum_below_prefix of the database does not apply, since its bound occurs only in a side condition, which would have to be a fact).

13.4 The proof

Of the 62 rule applications, those that carry the proof:

1. **The counters.** trace_merge_cases_ne joins the two cases of (68), both $i+1$, and counter_def makes $I = \mathsf{counter}(0,1,n)$, likewise $I' = \mathsf{counter}(0,1,n')$. In the context of the iteration k the simplifier reads i as k (earlier_counter), so the constraint (69) is $\mathsf{all}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{check}\ k \mapsto k < N)$, which invs_counter_upper_nonnegative resolves to $\mathsf{integer}(n) \le N$; with the exit $N \le i' = \mathsf{integer}(n)$ (current_counter) this gives $\mathsf{integer}(n) = N$, and for the second loop $\mathsf{integer}(n') = N$, hence $n' = n$ (integer_injective).
2. **The first loop is stores.** In the update of (67), $i \bmod 2 = 0$ is $k \bmod 2 = 0$ and the guards are $k \bmod 2 = 0$ and $0 < k \bmod 2$; stores_index_cases_def matches with $P := \lambda k.\ k \bmod 2 = 0$, $c := 2$, $d := 0$ (the higher-order P by abstraction of the guard), so $a' = \mathsf{current}(\mathsf{stores}(a_0, \lfloor\mathsf{counter}(0,1,n)\rfloor, \mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ k \bmod 2 = 0\ \mathsf{then}\ 2\ \mathsf{else}\ 0)))$.
3. **The read in the second loop.** The update of (70) reads $a'[i]$, i.e. $a'[k]$; current_stores makes it $\mathsf{if}\ \mathsf{occurs}(k, \lfloor\mathsf{counter}(0,1,n)\rfloor)\ \mathsf{then}\ \mathsf{earlier}(\mathsf{indexof}(k, \lfloor\mathsf{counter}(0,1,n)\rfloor), \mathsf{map}(\mathsf{for}\ \ldots\ \mathsf{yield}\ \ldots))\ \mathsf{else}\ a_0[k]$, occurs_counter_up the test $0 \le k < \mathsf{integer}(n)$, which the context of the iteration decides (it has $0 \le k < n' = n$), indexof_counter_up the index $\mathsf{natural}(k)$, and the read of the map at k is $\mathsf{if}\ k \bmod 2 = 0\ \mathsf{then}\ 2\ \mathsf{else}\ 0$. The second loop is now $\mathsf{trace}(s_0,\ \mathsf{for}\ \ldots\ \mathsf{do}\ s,k,i \mapsto \mathsf{case}\ k = 0 \Rightarrow s[0 := 0]\ \ \mathsf{case}\ 0 < k \Rightarrow s[0 := s[0] + (\mathsf{if}\ k \bmod 2 = 0\ \mathsf{then}\ 2\ \mathsf{else}\ 0)])$: a sum in a cell, with the summand a function of the iteration.
4. **The cell is a sum.** The postcondition reads $s'[0] = \mathsf{current}(S)[0]$; select_current_trace_reset_add matches with $j := 0$, $c := 0$ and $G := \lambda k.\ \mathsf{if}\ k \bmod 2 = 0\ \mathsf{then}\ 2\ \mathsf{else}\ 0$, abstracted from the summand, and yields $\mathsf{if}\ n = 0\ \mathsf{then}\ s_0[0]\ \mathsf{else}\ 0 + \mathsf{sum}(\mathsf{map}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ 0 < k\ \mathsf{then}\ G(k)\ \mathsf{else}\ 0))$; the case $n = 0$ is $\mathsf{integer}(n) = 0$ (zero_integer), false under $\mathsf{integer}(n) = N > 0$.
5. **The bound.** sum_cases_below matches the remaining atom $\mathsf{sum}(\ldots) \le 2N$ with $e := 2N$, $c := 2$, $d := 0$ and P the parity; its conditions $0 \le 2N$, $2\,\mathsf{integer}(n) \le 2N$ and $0 \le 2N$ hold by $\mathsf{integer}(n) = N$. The goal is closed.

A variant asserting `sum[0] <= N - 2` stays open with the residual $\neg(0 < N)$: the lemmas compute the value $N-1$ of the sum (the summand 2 at the even positions from 2 on) and the bound is false; the same lemmas close `brs3` to `brs5`, and sum_mod_below in place of the third closes `ms2` to `ms5`, whose arrays hold $i \bmod m$ (Section 14.5).

13.5 The idea

The scalar version of this program, which the Dafny translations of SV-COMP produce, is closed by current_trace_reset_add alone; the C program needs the same lemma for a cell of an array, with the same right-hand side. This is "name the normal form" of Section 18.1 at the level of the database: the shape of the data decides which lemma matches, and the lemma for the new shape should land in the normal form of the old, so that everything after it is shared. The bug that this test found on the way is finding 10 of the algorithms seen from the engine: the simplifier's eta reduction had turned the instance $\lambda z.\ a[z]$ of G into the array a itself, which left an application $a(k)$ that no rule about $a[k]$ matched, fixed in version 0.35.15 by restricting eta reduction to lambdas of array type.

14 An Update in Place and a Sum of Remainders

14.1 The program

The test `sum/conda` fills an array with ones, modifies it in place by cases on the old value, and sums it; the assertion is the exact sum.

```
sum[0] = 0;
for (i = 0; i < N; i++) a[i] = 1;
for (i = 0; i < N; i++) { if (a[i] == 1) a[i] = a[i] + 1; else a[i] = a[i] - 1; }
for (i = 0; i < N; i++) sum[0] = sum[0] + a[i];
__VERIFIER_assert(sum[0] == 2 * N);
```

14.2 The trace equation system

Three loops in sequence, each over its own counter of n, n', n'' steps, all equal to N by their tests and exits as in Section 13. The middle one is the interesting equation: its update reads the cell it writes, and its counter trace I' has the array among its dependencies, since the guards mention it,

$$A = \mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor\ \mathsf{do}\ a,k,i \mapsto a[i := 1]) \tag{71}$$

$$A' = \mathsf{trace}(a',\ \mathsf{for}\ \lfloor K'\rfloor, \lfloor I'\rfloor\ \mathsf{do}\ a,k,i \mapsto \mathsf{case}\ a[i]=1 \Rightarrow a[i := a[i]+1]\ \ \mathsf{case}\ a[i]\neq 1 \Rightarrow a[i := a[i]-1]) \tag{72}$$

$$I' = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K'\rfloor, \lfloor A'\rfloor\ \mathsf{do}\ i,k,a \mapsto \mathsf{case}\ a[i]=1 \Rightarrow i+1\ \ \mathsf{case}\ a[i]\neq 1 \Rightarrow i+1)$$

$$S = \mathsf{trace}(s_0[0 := 0],\ \mathsf{for}\ \lfloor K''\rfloor, \lfloor I''\rfloor\ \mathsf{do}\ s,k,i \mapsto s[0 := s[0] + a''[i]]) \tag{73}$$

$$\Longrightarrow\ \mathsf{current}(S)[0] = 2N$$

with $a' = \mathsf{current}(A)$ and $a'' = \mathsf{current}(A')$.

14.3 The lemmas

stores_cases_def :

$$\mathsf{trace}(a_0, \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n)\rfloor, \lfloor \mathsf{counter}(0,1,n)\rfloor\ \mathsf{do}\ a,k,i \mapsto \mathsf{case}\ P(a[k]) \Rightarrow a[k := c]\ \ \mathsf{case}\ \neg P(a[k]) \Rightarrow a[k := G(a[k])])$$
$$= \mathsf{stores}(a_0, \lfloor \mathsf{counter}(0,1,n)\rfloor, \mathsf{map}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ P(a_0[k])\ \mathsf{then}\ c\ \mathsf{else}\ G(a_0[k])))$$

Each position k is written once, in iteration k, and read only there before it is written, so the loop stores values computed from the *initial* array. The first case stores a constant and not $G'(a[k])$: a higher-order instance must contain the term it abstracts (`Rewrite.abstraction`), and $a[k] + 1$ is 2 in this program by the time the lemma is tried (step 2 below). The rest is the database's: stores_def and maps_const for the first loop, current_stores with occurs_counter_up, indexof_counter_up and earlier_consts for the reads, current_trace_add_at for the sum at the constant key, selects_stores_counter and sum_consts for the result.

14.4 The proof

Of the 69 rule applications:

1. **The first loop.** (71) is $\mathsf{stores}(a_0, \lfloor I \rfloor, \mathsf{map}(\mathsf{for}\ \lfloor K \rfloor\ \mathsf{yield}\ k \mapsto 1))$ by stores_def, and the map of a constant is $\lfloor \mathsf{consts}(n, 1) \rfloor$ (maps_const); I is $\mathsf{counter}(0, 1, n)$.
2. **The guards of the second loop.** In the context of the iteration k of (72), the previous value a at position k is unmodified, $a[k] = a'[k]$, and $a'[k]$ is read from the stores of the first loop: current_stores gives $\mathsf{if}\ \mathsf{occurs}(k, \lfloor I \rfloor)\ \mathsf{then}\ \mathsf{earlier}(\mathsf{indexof}(k, \lfloor I \rfloor), \lfloor \mathsf{consts}(n, 1) \rfloor)\ \mathsf{else}\ a_0[k]$, the test is $0 \le k < \mathsf{integer}(n)$ by occurs_counter_up, true in the iteration, and the read of the constant sequence is 1 (earlier_consts). So $a[k] = 1$ inside the update: the first result $a[k] + 1$ is 2, the guards are $a[k] = 1$ and $a[k] \neq 1$ on the *bound* variable a, which the lemma's P abstracts.
3. **The second loop is stores.** stores_cases_def matches (72) with $P := \lambda z.\ z = 1$, $c := 2$, $G := \lambda z.\ z - 1$ and gives $\mathsf{stores}(a', \lfloor \mathsf{counter}(0, 1, n') \rfloor, \mathsf{map}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n') \rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ a'[k] = 1\ \mathsf{then}\ 2\ \mathsf{else}\ a'[k] - 1))$; the read $a'[k]$ in the map is again 1 by step 2, the conditional is 2, and the map is $\lfloor \mathsf{consts}(n', 2) \rfloor$.
4. **The sum.** (73) is a trace that adds to the cell 0 in every iteration; current_trace_add_at makes $\mathsf{current}(S) = s_0[0 := 0][0 := 0 + \mathsf{sum}(\mathsf{map}(\mathsf{for}\ \lfloor K'' \rfloor, \lfloor I'' \rfloor\ \mathsf{yield}\ k, i \mapsto a''[i]))]$, the map of reads is $\mathsf{selects}(a'', \lfloor I'' \rfloor)$ (selects), and with $n'' = n' = n$ (integer_injective from the three exits) selects_stores_counter reads the stored values back at the positions of the counter: $\lfloor \mathsf{consts}(n, 2) \rfloor$, whose sum is $2\,\mathsf{integer}(n)$ (sum_consts). The postcondition $2\,\mathsf{integer}(n) = 2N$ holds by $\mathsf{integer}(n) = N$.

The tests `s1lif` to `s5lif`, whose second loops have the same shape with other cases, close by the same steps (69 rule applications for `s1lif`); a false assertion `sum[0] == 2 * N + 1` leaves $2\,\mathsf{integer}(n) = 2N + 1$, i.e. the lemmas compute the sum rather than close the goal.

14.5 A sum of remainders

The test `sum/ms2` stores $i \bmod 2$ at i and asserts `sum[0] <= N` for the sum of Section 13. The first loop is plain stores of a map, $\mathsf{stores}(a_0, \lfloor I \rfloor, \mathsf{map}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{yield}\ k, i \mapsto \mathsf{if}\ 0 \le i\ \mathsf{then}\ i \bmod 2\ \mathsf{else}\ -(-i \bmod 2)))$ (the remainder of C under a sign that the context of the map does not know yet), the read $a'[k]$ in the second loop goes through current_stores to the map at k, where $0 \le k$ holds and the read is $k \bmod 2$, select_current_trace_reset_add abstracts $G := \lambda k.\ k \bmod 2$, and

$$\mathsf{sum_mod_below}: \quad 0 < m \ \wedge\ \mathsf{integer}(n) \cdot (m - 1) \le e$$
$$\implies \mathsf{sum}(\mathsf{map}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n) \rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ 0 < k\ \mathsf{then}\ k \bmod m\ \mathsf{else}\ 0)) \le e \iff \mathsf{true}$$

closes the bound with $m := 2$, $e := N$ and $\mathsf{integer}(n) = N$ (61 rule applications). The three tests `brs2`, `conda` and `ms2` share every step but the last two: the shape of the summands (constants by index, constants by value, remainders) selects the introduction lemma for the stores and the bound lemma for the sum.

14.6 The idea

Loops in sequence compose through the current values, as in the counting sort (Section 9): the stores of one loop are read in the update of the next, where the context of the iteration decides the membership tests, and the read collapses to the stored value before the introduction lemma of the next loop is tried. The order matters: stores_cases_def sees $a[k] + 1$ as 2 because the read was resolved first, which is why its first case is a constant; a lemma written from the program text, with $G'(a[k])$ in the first case, would not match the simplified update. The lemmas for a program are written for the goal as the simplifier prints it, not for the program (the pattern of Section 18.2, arithmetic in the goal's form).

15 Strided Writes: Parity, Division, and a Case the Simplifier Does Not Close

15.1 The program

The test `forall/pr2` fills an array of even size pairwise with values that depend on a bound `MINVAL`, which the loop does not change, and asserts that every cell is at least the bound or zero.

```
if (CELLCOUNT % 2 != 0) return 1;
for (i = 1; i <= CELLCOUNT / 2; i++) {
  volArray[i*2 - 2] = (3 >= MINVAL) ? 3 : 0;
  volArray[i*2 - 1] = (1 >= MINVAL) ? 1 : 0;
}
for (i = 0; i < CELLCOUNT; i++)
  __VERIFIER_assert(volArray[i] >= MINVAL || volArray[i] == 0);
```

The run is `-unbounded -teq -simplify -split -simplify -include teq:strides -include benchmarks/monocera/lemmas.dfy`: the module `strides` holds the lemmas on strided stores, and `lemmas.dfy` the one lemma that is not in the database, half_even.

15.2 The trace equation system

The loop runs from $i = 1$, so its trace of i is $\mathsf{counter}(1,1,n)$ and the stores are at $2k$ and $2k+1$ for the iteration $k = i - 1$. The body has four paths, two conditionals in sequence, of which the simplifier's context removes the impossible ones ($\texttt{MINVAL} \le 1$ implies $\texttt{MINVAL} \le 3$), and the merged update has three cases on the bound:

$$V = \mathsf{trace}(v_0, \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor, \lfloor\mathsf{counter}(1,1,n)\rfloor\ \mathsf{do}\ v,k,i \mapsto \\ \mathsf{case}\ M \le 1 \Rightarrow v[2k := 3][2k+1 := 1]\ \ \mathsf{case}\ 1 < M \le 3 \Rightarrow v[2k := 3][2k+1 := 0] \\ \mathsf{case}\ 3 < M \Rightarrow v[2k := 0][2k+1 := 0]) \tag{74}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{check}\ k \mapsto k < C/2) \qquad 1 + \mathsf{integer}(n) = i' \qquad \neg(i' \le C/2) \tag{75}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K'\rfloor, \lfloor I'\rfloor\ \mathsf{check}\ k,i \mapsto v'[k] \ge M \vee v'[k] = 0) \qquad i'' = \mathsf{integer}(n') < C \tag{76}$$

$$\implies v'[i''] \ge M \vee v'[i''] = 0$$

with C for `CELLCOUNT`, M for `MINVAL`, $v' = \mathsf{current}(V)$, and $C \bmod 2 = 0$, $1 < C$ from the harness. The constraint (75) is resolved by invs_counter_upper to $n = 0 \vee (0 < C/2 \wedge \mathsf{integer}(n) - 1 < C/2)$, and with the exit $C/2 < 1 + \mathsf{integer}(n)$ this is the fact $\mathsf{integer}(n) = C/2$: the number of iterations is the half of the size, as a quotient.

15.3 The lemmas

$$\mathsf{select_current_trace_stride2_cases}: \quad R \Leftrightarrow (\neg P \wedge \neg Q) \\ \implies \mathsf{current}(\mathsf{trace}(a_0, \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor, \lfloor\mathsf{counter}(1,1,n)\rfloor\ \mathsf{do}\ a,k,i \mapsto \\ \mathsf{case}\ P \Rightarrow a[2k := X_1][2k+1 := Y_1]\ \ \mathsf{case}\ Q \Rightarrow a[2k := X_2][2k+1 := Y_2] \\ \mathsf{case}\ R \Rightarrow a[2k := X_3][2k+1 := Y_3]))[j] \\ = \mathsf{if}\ 0 \le j < 2\,\mathsf{integer}(n) \\ \mathsf{then}\ (\mathsf{if}\ j \bmod 2 = 0\ \mathsf{then}\ (\mathsf{if}\ P\ \mathsf{then}\ X_1\ \mathsf{else\ if}\ Q\ \mathsf{then}\ X_2\ \mathsf{else}\ X_3) \\ \mathsf{else}\ (\mathsf{if}\ P\ \mathsf{then}\ Y_1\ \mathsf{else\ if}\ Q\ \mathsf{then}\ Y_2\ \mathsf{else}\ Y_3)) \\ \mathsf{else}\ a_0[j]$$

$$\mathsf{div_nonpos}: \quad 0 < c \implies (x/c \le 0) \Leftrightarrow (x < c)$$

$$\mathsf{half_even}: \quad x \bmod 2 = 0 \implies (n = x/2) \Leftrightarrow (x = 2n)$$

The stride lemma reads a cell of the final array: the pairs $2k, 2k+1$ are disjoint and cover $0, \ldots, 2n-1$, outside of which the initial array remains, and the value is chosen by the parity of the cell and by the guards, which do not depend on the iteration. Its side condition requires the guards to be

exhaustive; without it the update would be unspecified where none holds and the equation unsound (the guards of a trace need not be complete since `break`, Section 3 of the paper as implemented). The lemma fixes two stores and three cases on its left-hand side, as the summaries of the algorithms fix the fragment structure of one program; `pr3` to `pr5`, with 2^s combinations of s per-cell guards, have no lemma.

15.4 The proof

Of the 81 rule applications over the goals of `-split`:

1. **The reads.** select_current_trace_stride2_cases fires four times, on the read $v'[k]$ in the constraint (76) of the checking loop and on $v'[i'']$ in the conclusion (each twice, in the two halves of the disjunction), with $P := M \leq 1$, $Q := 1 < M \leq 3$ and the six values; the side condition holds since the simplifier produced the three guards as a case split on M. Each read is now a conditional of constants below $2\,\mathsf{integer}(n)$, every branch of which satisfies $x \geq M \vee x = 0$ under its guard, so the assertion holds for $0 \leq i'' < 2\,\mathsf{integer}(n)$.
2. **The harness.** `-split` separates the case of no iteration, $C/2 \leq 0$, in which the array is the initial one; div_nonpos turns $C/2 \leq 0$ into $C < 2$, which contradicts $1 < C$, three times (one per goal that carries the case).
3. **The cells beyond the stores.** What remains is $2\,\mathsf{integer}(n) \leq i'' < C$ with $\mathsf{integer}(n) = C/2$ and $C \bmod 2 = 0$: an empty range, which the simplifier does not see, since it inlines solutions of arithmetic equations into polynomials only and $C/2$ is an atom to it, not a polynomial in C. Without half_even this goal remains (the run with the module `strides` alone leaves exactly it, with the constraint of the checking loop fully resolved). half_even rewrites the fact $\mathsf{integer}(n) = C/2$ under $C \bmod 2 = 0$ into $C = 2\,\mathsf{integer}(n)$, a polynomial equation, which the simplifier solves and substitutes: $2\,\mathsf{integer}(n) \leq i'' < 2\,\mathsf{integer}(n)$ is false.

The constant strides without cases, select_current_trace_stride2 and stride3, close two further tests in the same way and without the harness,
`array_doub_access_init_const` (stride 2) and `array_tripl_access_init_const` (stride 3).
In the first, $N = 100000$ is a literal, the loop `i <= N` is a counter of n steps with $\mathsf{integer}(n) = 100001$ decided from the constraint and the exit, and the read $a'[k]$ for $k \leq 200000$ is

$$\mathsf{if}\ 0 \leq k < 2\,\mathsf{integer}(n)\ \mathsf{then}\ (\mathsf{if}\ k \bmod 2 = 0\ \mathsf{then}\ 0\ \mathsf{else}\ 0)\ \mathsf{else}\ a_0[k]$$

(44 rule applications).

15.5 The idea

Three obstacles of different kinds in one small program. The stride is a shape of the update, answered by a lemma that fixes the shape; the quotient in the bound is an obstacle of the simplifier's arithmetic, answered by a lemma that restates the fact in the form the decision procedure handles (the pattern "arithmetic in the goal's form" of Section 18.2, here the form the *engine* wants); and the exhaustiveness of the guards is a soundness premise that the lemma must state, since the engine does not. half_even stays outside the database as a workaround for a documented limitation (TODO.md: solutions are inlined into polynomials only), the proper fix being in the simplifier.

16 Stores Ahead: Upwards Proved, Downwards Diagnosed

16.1 The programs

The test `forall/rewnif` writes one cell ahead before it writes the current one; `rewnifrev` (under `forall` and `min`) does the same in a loop that runs downwards. Both assert `a[i] >= i` for all cells.

```
for (i = 0; i < SIZE; i++) { if (i + 1 < SIZE) a[i+1] = i; a[i] = i; }        // rewnif
for (i = SIZE - 1; i >= 0; i--) { if (i - 1 >= 0) a[i-1] = i - 2; a[i] = i; }  // rewnifrev
for (i = 0; i < SIZE; i++) __VERIFIER_assert(a[i] >= i);
```

The runs use the module `strides` (`-include teq:strides`), which holds the lemmas on stores ahead. The summary of the MonoCera tests records `rewnif` as closed and `rewnifrev` as the one test whose lemma "has the same form as the trace in the goal when printed, yet does not match", with the literal -1 excluded as the cause. This section re-traces the first and resolves the second.

16.2 Upwards: the trace equation system and the proof

With S for `SIZE`, $1 < S$ from the harness, the filling loop is

$$A = \mathsf{trace}(a_0, \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ a, k, i \mapsto \mathsf{case}\ i+1 < S \Rightarrow a[i+1 := i][i := i]\ \ \mathsf{case}\ \neg(i+1 < S) \Rightarrow a[i := i]) \tag{77}$$

$$I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ i, k \mapsto \mathsf{case}\ i+1 < S \Rightarrow i+1\ \ \mathsf{case}\ \neg(i+1 < S) \Rightarrow i+1)$$
$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor A \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, a, i \mapsto i < S)$$

and the lemma of the module is

$$\begin{aligned}
&\mathsf{select_current_trace_ahead}: \quad \mathsf{integer}(n) \le m \\
&\quad \Longrightarrow\ \mathsf{current}(\mathsf{trace}(a_0, \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n) \rfloor, \lfloor \mathsf{counter}(0,1,n) \rfloor\ \mathsf{do}\ a, k, i \mapsto \\
&\qquad\quad \mathsf{case}\ k+1 < m \Rightarrow a[k+1 := F(k)][k := G(k)]\ \ \mathsf{case}\ m = k+1 \Rightarrow a[k := G(k)]))[j] \\
&\qquad = \mathsf{if}\ 0 \le j < \mathsf{integer}(n)\ \mathsf{then}\ G(j) \\
&\qquad\quad \mathsf{else\ if}\ j = \mathsf{integer}(n) \wedge 0 < j < m\ \mathsf{then}\ F(j-1)\ \mathsf{else}\ a_0[j]
\end{aligned}$$

The value written ahead at $k+1$ is overwritten by the next iteration, so a cell $j < n$ holds what iteration j writes last, and the cell n holds what the last iteration wrote ahead, if it did. The side condition $\mathsf{integer}(n) \le m$ makes the two guards cover all iterations: for $k < n \le m$, either $k+1 < m$ or $k+1 = m$. The proof takes 27 rule applications: the counter I is merged and recognized, the constraint gives $\mathsf{integer}(n) \le S$, which is the side condition, and $\mathsf{select_current_trace_ahead}$ fires twice on the reads $a'[k]$ of the checking loop's constraint and $a'[i'']$ of the conclusion, with $m := S$ and $F := G := \lambda z.\ z$ (the stored values i abstracted over the bound variable), giving $\mathsf{if}\ 0 \le j < \mathsf{integer}(n)\ \mathsf{then}\ j\ \mathsf{else\ if}\ j = \mathsf{integer}(n) \wedge 0 < j < S\ \mathsf{then}\ j-1\ \mathsf{else}\ a_0[j]$; the exit $i'' = \mathsf{integer}(n'')$ with $\mathsf{integer}(n'') = S = \mathsf{integer}(n)$ puts the conclusion's index in the first branch ($\mathsf{integer_injective}$), and $j \ge j$ holds.

16.3 Downwards: the goal that remained

For `rewnifrev` the counter runs from $S-1$ down, and the simplifier's goal, after 54 rule applications and with the module's lemma for the downward loop present but silent, is

$$A = \mathsf{trace}(a_0, \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ a, k, i \mapsto \mathsf{case}\ 1 \le i \Rightarrow a[i + (-1) := i + (-2)][i := i]\ \ \mathsf{case}\ i < 1 \Rightarrow a[i := i]) \tag{78}$$

$$a' = \mathsf{current}(A)$$
$$I = \mathsf{counter}(\mathsf{integer}(n) + i', -1, n) \qquad 0 = i' + 1 \qquad 0 < \mathsf{integer}(n) + i' < 99999 \tag{79}$$
$$\mathsf{above_iterations}(n'', \mathsf{map}(\mathsf{for}\ \lfloor K'' \rfloor\ \mathsf{yield}\ k \mapsto a'[k] + 1)) \tag{80}$$
$$i'' = \mathsf{integer}(n'') \le \mathsf{integer}(n) + i'$$
$$\Longrightarrow\ i'' \le a'[i'']$$

Two things have happened to the facts. The exit $\neg(0 \le i')$ with the constraint $0 \le i$ gives $i' = -1$, and the simplifier has eliminated S in favour of the counter: $S - 1 = \mathsf{integer}(n) + i'$, so the start of I is the term $\mathsf{integer}(n) + i'$ (with $i' = -1$ kept as an equation, not substituted into it). And the constraint of the checking loop, $\mathsf{all}(\mathsf{for}\ \ldots\ \mathsf{check}\ k \mapsto k \le a'[k])$, has been abstracted by $\mathsf{above_iterations_def}$ of the database into the atom (80) before the read $a'[k]$ was resolved, since no lemma resolved it: the fact is hidden from any later rule on $a'[k]$ (finding 3 of the algorithms, the lemma that "swallows" $k < t$ constraints). In `rewnif`, the read was rewritten first and the constraint resolved by itself.

16.4 The diagnosis

The module's lemma is select_current_trace_ahead mirrored: the dependencies $\lfloor \mathsf{counter}(0, 1, n) \rfloor$, $\lfloor \mathsf{counter}(s, -1, n) \rfloor$, the cases case $1 \leq i \Rightarrow a[i + (-1) := F(i)][i := G(i)]$ and case $i < 1 \Rightarrow a[i := G(i)]$, and the right-hand side $G(j)$ for $s - \mathsf{integer}(n) < j \leq s$, $F(j+1)$ at $j = s - \mathsf{integer}(n)$ if $0 \leq j$ and $0 < n$, else $a_0[j]$. Four variants were run on the test with the lemma injected as a user file:

variant	step of the counter	position of the store ahead	result
as in the module	literal -1	$i + (-1)$	does not fire
a	literal -1	$i - 1$	does not fire
b	variable d, condition $d = -1$	$i + (-1)$	does not fire
e	literal -1	$P(i)$, pinned by $\forall i.\ P(i) = i - 1$	does not fire
d	variable d, condition $d = -1$	$P(i)$, pinned	**fires twice, closed**

Both changes are needed, and both are patterns of Section 18.2 that the algorithms had established: a literal -1 in a counter pattern is not matched (principle 8 of `NOTES.md`, `reverse.dfy`, whose counter down has its step as a variable with $d = -1$ as a side condition), and arithmetic on a bound variable in a left-hand side, here the position $i - 1$ of the store, is not matched and is pinned (Section 7, the swaps at $j - 1$ of the insertion sort). The summary had excluded the literal -1 because the database matches $\mathsf{counter}(i, -1, n)$ elsewhere; it does so as a term, not as a dependency pattern matched through the context of the goal, which is where it stands here. With variant d the test closes with $s := \mathsf{integer}(n) + i'$, $F := \lambda z.\ z - 2$, $G := \lambda z.\ z$, also under `min`; a false assertion `a[i] >= i + 1` leaves a goal. The variant does not cover `rewnifrev2`, which stores ahead *upwards* ($a[i + 1]$) in a loop that runs downwards, a third shape, whose cell j holds the value stored ahead by the iteration below it.

16.5 The idea

The database lemma is right and the engine applies it to the upward loop; the downward loop fails on two literal patterns whose workarounds the algorithms had found in another guise. The lesson for the database is that lemma statements must follow the matcher's patterns as much as the goal's forms: steps of counters as variables with side conditions, positions computed from the bound variable as pinned higher-order variables. The lesson for the engine is the same as finding 12 and finding 34 of the algorithms: a match that fails without a visible reason costs more than its cause deserves, and a flag that reports which part of a left-hand side did not match would have answered this in one run. The restated lemma is a candidate for the module `strides`, which this document does not change.

17 A Recurrence on the Previous Cell (open)

17.1 The program

The test `forall/ifeqn2`: two arrays filled by recurrences on the previous cell, and the closed form asserted for every position.

```
for (i = 0; i < N; i++) { if (i == 0) a[0] = 4; else a[i] = a[i-1] + 4; }
for (i = 0; i < N; i++) { if (i == 0) b[0] = 1; else b[i] = b[i-1] + a[i-1]; }
for (i = 0; i < N; i++) __VERIFIER_assert(b[i] == 2*i*i + 2*i + 1);
```

17.2 The goal that remains

After 127 rule applications, all on counters and their reads, the goal is

$$A = \mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor\ \mathsf{do}\ a,k,i \mapsto \mathsf{case}\ k=0 \Rightarrow a[0:=4]\ \ \mathsf{case}\ 0<k \Rightarrow a[k:=a[k-1]+4]) \tag{81}$$

$$a' = \mathsf{current}(A)$$

$$B = \mathsf{trace}(b_0,\ \mathsf{for}\ \lfloor K'\rfloor, \lfloor I'\rfloor\ \mathsf{do}\ b,k,i \mapsto \mathsf{case}\ k=0 \Rightarrow b[0:=1]\ \ \mathsf{case}\ 0<k \Rightarrow b[k:=a'[k-1]+b[k-1]]) \tag{82}$$

$$b' = \mathsf{current}(B)$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K''\rfloor, \lfloor I''\rfloor\ \mathsf{check}\ k,i \mapsto b'[k] = 2k^2+2k+1) \tag{83}$$

$$i'' = \mathsf{integer}(n'') < N$$

$$\implies b'[i''] = 2\,i''^2 + 2\,i'' + 1$$

with $K = I = K' = I' = \mathsf{counter}(0,1,n)$, $\mathsf{integer}(n) = N$, and $K'' = I'' = \mathsf{counter}(0,1,n'')$ the counter of the checking loop. The two filling loops are not resolved: a store at k of a value read at $k-1$ is not a stores of a map (the value depends on the trace, not on the initial array), and no lemma of the database names such a recurrence. The checking loop has become the constraint (83), the assertion for the earlier iterations, and the goal is the assertion at the current index: an induction step without a closed form of b'.

17.3 What is missing

This is the first of the five classes of open goals that the summary of the MonoCera tests lists (recurrences on earlier entries with a polynomial closed form: `ifeqn1` to `ifeqn5`, `ifcomp`, `pcomp`, `sqm`, `nsqm`, `condm`), and TODO.md records that even a lemma written for the shape,

$$\mathsf{current}(\mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{do}\ a,k \mapsto a[k:=a[k-1]])) = \ldots,$$

does not match its instance: $k-1$ as a store index is arithmetic on a bound variable, the same obstacle as the swaps at $j-1$ of the insertion sort, where the pin of Section 7 is the way out (a higher-order $P(k)$ for the position, pinned to $k-1$). What a proof would need beyond the match:

- an introduction lemma that names the recurrence, e.g. $\mathsf{recur}(a_0,c,n)$ for $a[k] = a[k-1]+c$ from $a[0]=c$, with the acceleration $\mathsf{recur}(\ldots)[j] = c\,(j+1)$ for $0 \le j < n$: the closed form of an arithmetic progression, which is what $a'[k] = 4k+4$ needs;
- the same for b, whose increment is the entry of another named trace, i.e. a sum of the progression, $b'[k] = 1 + \sum_{j<k}(4j+4) = 2k^2+2k+1$: either a lemma on sums of affine maps (the database has sum over counters, not over maps of a counter with a polynomial body), or a recurrence solver as a lemma generator for the arithmetic fragment, which the ideas on the paper propose as related work (compositional recurrence analysis, closed forms for numerical loops);
- alternatively, the checking loop as an induction: (83) is the invariant at the earlier positions, and the goal at i'' follows from $b'[i''] = a'[i''-1] + b'[i''-1]$ and the constraint at $i''-1$, an instantiation at a witness with the recurrence unfolded once, which no rule of the database performs (the step from a constraint to a position is the pattern of Section 1, but here the position's value is given by the recurrence, not by a named function).

Neither Cuvée nor MonoCera proves this test (nor TriCera, CPAchecker or SeaHorn in the paper's evaluation): the instrumentation of the quantified assertion by a ghost variable still needs the polynomial invariant of the filling loops. A lemma-based proof needs the closed forms, which is the representation that trace equation systems are meant to provide, and a database that does not yet have them.

18 Principles

This section collects what the examples have in common: the shape that every proof takes, the rules that the matcher imposes on a lemma written for a program, the discipline that keeps such lemmas

sound and the rewriting terminating, the diagnostics that located each obstacle, and what the same principles look like on the C programs of the MonoCera benchmarks (`benchmarks/monocera`). The sources are the numbered findings of `NOTES.md`, the experiment threads kept in `pending/` (preludes, programs with wrong postconditions, scripts and logs, Oct 10 and 11, 2026), `doc/teq.md`, and the summary and case studies of the MonoCera tests. The state of the folder on Oct 12, 2026: 27 of its 30 files verify with `-teq -simplify` (nine of them with `-include teq:nested`); open are `is_sorted.dfy` (one substitution of the exit into an argument of sorted), the existential postcondition of `max_element.dfy` (no witness), and the lower bound (Section 11, blocked by the engine).

18.1 The shape of a proof

Every verified example is closed by lemmas of the following kinds, in this order from generic to specific. The first two kinds are the database's, the others are the algorithm's knowledge, stated at the beginning of its file.

1. **The database resolves single loops.** Counters, sums, stores, selects, counts, minima and maxima are introduced and accelerated by the modules of `-teq` without anything written for the program (the four loops of the counting sort up to the table of start positions, Section 9).
2. **Fragment lemmas resolve inner counters.** Theorem 4.8 of the paper turns the trace equation system of an inner loop into fresh sequences inside the iteration constraint of the outer loop, which the simplifier distributes into fragments over incomparable dependency lists that do not see each other's facts (the two minimal problems, `invs_counter_equation.dfy` and `invs_fragments.dfy`). The module `nested` does the resolution by rewriting, one fragment at a time in the order in which `-teq` produces them: the inner trace is a counter, its final value is its length, the test bounds the length, the exit fixes it, the numbers of inner iterations are a constant sequence (Sections 2 and 4.3). The same lemmas serve every nested example.
3. **A summary states what an inner loop computes.** Its left-hand side is the outer trace, read through the current value, or the whole atom of the postcondition; its side conditions are the fragments, matched against the facts; its right-hand side is a trace of an abstract step, argmin swaps, insert, shake, ¬nodup (Sections 4.4, 6, 10). An inner result stored at the outer position needs no summary but a chain of specific substitutions (Section 3).
4. **An induction lemma states the invariant of the outer loop** as a quantified side condition over the abstract step, which the simplifier decides by step lemmas on atoms; the invariant is a predicate indexed by the iteration (bubbled, shaken, the sorted prefix), and the lemma constrains all steps of the trace (Sections 4.5 and 5). This is the scheme of current_trace_bounded of the database: no invariant is inferred, its shape is the lemma's knowledge, and the database proves the step.
5. **Rules on the atoms of the postcondition** characterize a named result: the bounds of argmin and lb, the regions of a partition, the counts of merged, the two cases of the exit for distincts. The postcondition is never rewritten as a whole; each atom finds its rule, with the exit of the loop among the facts.
6. **Name the normal form.** The counter of a loop is not inlined into the facts, so a count over $\lfloor I \rfloor$ and over $\lfloor \mathsf{counter}(0, 1, n) \rfloor$ differ (`count_occurrences.dfy`); a lemma that names the term makes both sides equal. Quantified preconditions are stated as atoms of the specification (keys_below, Section 9), postconditions in the database's vocabulary (occurs, distincts, count over selects) where the standard form has no witness or no matchable shape.

A data-dependent number of iterations (merge, gnome sort, the early exit of the first duplicate) is no obstacle to this shape as long as the property can be stated on the final values and the exit is read from the facts; what the paper's loop shapes exclude is the characterization of the traces, not the postcondition (Section 8).

pattern	found on	finding
specific left-hand side of an all rule	cocktail sort, nested mult	15, 29
fresh variables for every all side condition	bubble sort, cocktail sort	9, 27
open trace variables for the traces of the goal	merge, counting sort, first duplicate	14, 31
the pin for literal patterns	merge, insertion sort, gnome sort	14
the whole atom as left-hand side	merge, gnome sort	16
arithmetic in the goal's form	cocktail sort, bubble sort	11, 28
side conditions in the forms of the included modules	insertion sorts, cocktail sort	22, 37
no condition on the rule's own term	lower bound, bubble sort	20, 32
the exit of an outer loop as two implications	first duplicate	31
atoms for constraints and quantified hypotheses	linear search, counting sort	13, 19
variables of the method named apart	fragments, invs_fragments	9, 21

Table 1: The patterns of Section 18.2, the examples that forced them, and the findings of `NOTES.md` that record them.

18.2 Writing a lemma that the matcher accepts

The matcher of the paper (Section 5.2) relates the binding structure of combinators, follows the definitions of the context on demand, synthesizes dependencies that are variables of the rule as maps, and matches side conditions against the facts while they have an open variable. From these four mechanisms follow the patterns below, each found by an example that did not close without it (Table 1).

1. **A specific left-hand side.** An all pattern whose body has a bare bound variable of a variable dependency on one side, $n \leq j$ or $j = \mathsf{integer}(nn)$, matches every comparison, respectively equation, fragment: the dependency is synthesized as a map of the fact's dependencies. With an all side condition such a rule cycles (Section 18.4). A function symbol on the side of the bound variable, $\mathsf{integer}(nn) \leq n$ or $J = \mathsf{counter}(0, 1, nn)$, an all with a specific body, or an atom of the specification is specific.
2. **The whole atom as left-hand side.** A variable argument of an uninterpreted function is not expanded by its definition when the simplifier rewrites below it: $\mathsf{sorted}(c, 0, n)$ with $c = \mathsf{current}(C)$ is not reached by a rule on $\mathsf{current}(\mathsf{trace}(\ldots, \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots))$, but $\mathsf{sorted}(\mathsf{current}(\mathsf{trace}(\ldots, \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots)), 0, s)$ is, and the length is a variable s with $s = m + n$ as a side condition, since a pattern $m + n$ of two variables is not matched (Section 7).
3. **Fresh variables for every all side condition.** A condition is matched against the facts, with backtracking, while a variable of it is open; once all are bound it is closed and decided by the simplifier, which does not find a closed all among the facts (these carry unused dependencies, which the simplifier also drops from the instance, and their bound variables are renamed). So each all condition binds a fresh sequence, $\mathsf{all}(\mathsf{for}\ ks, js_6, Js_6\ \mathsf{check}\ k, j, J \mapsto j = \mathsf{current}(J))$, and an equation $js_6 = js_4 \wedge Js_6 = Js_3$ identifies it afterwards; the equation is closed and trivially decided. The 21st condition of the cocktail summary stayed closed for a day (Section 6).
4. **Open trace variables.** The traces of the goal are variables K, I, R with their equations as hypotheses. $\lfloor \mathsf{counter}(0, 1, m) \rfloor$ as a *dependency* of a pattern matches $\lfloor K \rfloor$ through the context; as an *argument* of a function it never does, and a closed equation $is = \lfloor \mathsf{trace}(\ldots, \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots) \rfloor$ is decided and fails, since $\lfloor I \rfloor$ is not expanded. A rule takes $Kv = \mathsf{counter}(0, 1, m)$ or $Iv = \mathsf{trace}(0, \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots)$ as an *open* condition, matched against the hypothesis, and uses $\lfloor Kv \rfloor$, $\lfloor Iv \rfloor$ afterwards (Sections 7, 9, 10).
5. **The pin.** A literal pattern that the matcher does not handle, a disjunctive guard, the position $j - 1$ of a store, the initial value $n - k - 2$ of a counter, the step -1, becomes a higher-order variable $G(i, j)$, $P(j)$, $X(k)$, d, which matches anything, and a quantified side condition $\forall i, j.\ G(i, j) \Leftrightarrow \varphi$ or an equation pins it to the intended term; the simplifier decides the pin by beta reduction. A pin must mention only variables that the left-hand side or an earlier condition binds, and it must follow the fragments: a test $0 < j \wedge y[j] < y[j - 1]$ is split into two constraints, so its pin is two pins (`insertion_sort.dfy`). A higher-order variable

for a bare array read $a[i]$ is instantiated by the array itself (finding 10), so reads are pinned, as $U(i, j) = a[i]$ in the merge, or written literally.

6. **Arithmetic in the goal's form.** The decision procedure of the context combines difference constraints; $len \leq 2\,\mathsf{integer}(m) + 1$ is not derived from $n \leq \mathsf{integer}(m) + lo + 1$ and $lo = \mathsf{integer}(m)$. A condition states the bound as the goal has it, with the program's variable as an open rule variable matched against its equation ($l = \mathsf{integer}(m)$, then $len \leq \mathsf{integer}(m)+l+1$); a length $len - k - 1$ that a pattern cannot contain is a variable p with $p = len - k - 1$ as a closed condition, decided once p is bound (Section 5). Arithmetic on a bound variable is admissible on right-hand sides and in side conditions, never in a left-hand side.
7. **Side conditions in the forms of the included modules.** The fragments are rewritten by the modules before a summary sees them, so a summary is written for the normal form of the whole database that the run includes: $j = \mathsf{current}(J)$ without `teq:nested`, $j = \mathsf{integer}(nn) + k + 1$ with it, as the goal prints them. A file includes exactly the modules its lemmas are written for; all nested loops of the folder are now written for `teq:nested`, so that the module can become part of `-teq` (finding 37).
8. **No condition on the rule's own term.** The instance of a side condition is simplified before it is decided or matched, so a condition $l = \mathsf{current}(Lv)$ of a rule on $\mathsf{current}(Lv)$ with Lv bound rewrites $\mathsf{current}(Lv)$ by the rule itself and does not terminate. The summaries take $\mathsf{current}(\mathsf{results}(\ldots))$ as left-hand side rather than $\mathsf{results}(\ldots)$ for the same reason, and the exit of a loop is read from the facts ($x = \mathsf{current}(Iv)$ open, then $n \leq x$, Section 8) or abstracted by an atom, never stated on the current value of the trace being rewritten (Section 11).
9. **The exit of an outer loop.** The exit of an inner loop is a fragment and a side condition like the others. The exit of the outer loop stands in the conclusion of the goal, $\neg exit \vee post$, which `-split` does not split when the exit is a disjunction, and a rule on an atom of *post* sees neither literal; a postcondition stated as two implications gives each goal its literal (Section 10). The case of no iteration appears only after a first simplification, hence `-simplify -split -simplify` where it matters (`is_sorted.dfy`, `reverse.dfy`).
10. **Atoms for iteration constraints and quantified hypotheses.** An iteration constraint is never a side condition (Section 18.4); a definition lemma abstracts it by an atom with a body that the matcher cannot synthesize, an explicit read $\mathsf{earlier}(\mathsf{natural}(k), js) = c$, and the lemma that needs the constraint takes the atom (Section 1). A quantified hypothesis is matched by no side condition, open or closed; the specification provides the atom (Section 9).
11. **Variables named apart.** Rule variables are not renamed apart from the variables of the goal. A method variable with the name of a rule variable of an open condition makes the condition coincide with the fact literally; it is decided true without binding the variable, and the run ends with "variables of a rule not instantiated by its match" on stderr, which *replaces the goal*: zero goals with an error is not a proof. The fragments of `-teq` are named $K?_{14}$ and never clash; test methods and pins name their variables KK, NN, len apart.
12. **Dependencies.** Lemmas carry the extra dependencies that the simplifier keeps in maps (the counter of the loop as hs), the matcher does not backtrack over the selection of dependencies when a side condition fails (hence $\mathsf{invs_current_counter_other}$), equal counters $K = I$ of one loop are not identified, and a higher-order summand $X(k)$ matches a single summand only, which is why the resolved forms are written literally.

18.3 Soundness

The lemmas of the examples are axioms: the abstract predicates sorted, perm, $\mathsf{partitioned}$, $\mathsf{bubbled}$, shaken and the functions argmin, insert, shake, nodup, merged, lb have no definitions, and the summaries state the correctness of an inner loop in one line. Nothing in the tool checks them, so the discipline of the folder replaces the check.

- **Every result is checked on a wrong postcondition.** Each thread of `pending/` has a `program_wrong.dfy` ($\mathsf{sorted}(b, 0, n + 1)$, the sum plus one, the negated flag); a prelude that closes its goal is unsound or vacuous, not a success. Two such cases occurred: a rule

$\mathsf{nodup}(a, k, n) \Leftrightarrow \mathsf{true}$ under the hypothesis $\mathsf{distincts}(\ldots)$ fired inside the context in which a side condition of a rule *on* $\mathsf{distincts}$ was matched, so the atom was derived from itself (finding 13); and a condition with an open dependency and an equation on its bound variable was decided vacuously (`vac.dfy`). The lesson: a side condition must not be simplifiable by rules that use the conclusion, and the safe form of a condition is an atom that is matched literally.

- **A lemma must be a lemma.** The lower bound closes with an exit $h \leq l$ of two open variables matched against the fact $hi \leq lo$ (`lb_cmp0.dfy`); in this goal its conclusion is right, since the exit holds in the final state, but h and l are not tied to the traces and the rule is not a theorem. Such a rule is not admitted, and the file stays open (Section 11).
- **An induction constrains all steps.** A quantified condition over $0 \leq k < n$ with $n \leq \mathsf{integer}(m)$ steps leaves the further steps unconstrained. The number of steps is fixed exactly ($n = \mathsf{integer}(m)$, a fact of the goal), or the invariant is clamped so that every step keeps it ($\mathsf{bubbled}$ for $k \geq len$), or the result holds for any number of iterations beyond a bound ($m + n \leq \mathsf{integer}(nn)$ in the merge).
- **Soundness premises are the matcher's facts.** The Lean proofs of the core modules showed the premises that the lemmas need and the statements omit: dependencies of equal length, indices within the lists, complete guards; the fallback encoding does not assume them. The modules `nested`, `sorting` and `strides` have no Lean proofs yet, and the abstract predicates would need definitions there for the step lemmas to be provable; both are open items.
- **A killed run prints no goal.** A runner that counts goals reports a timeout as a proof: the first "closed" runs of `row_sums`, `transpose` and most variants of the lower bound were kills at 120 or 300 s (finding 18). Every status of the folder was re-established with the exit status read, and long runs use a private copy of the jar, since a rebuild during a run crashes the JVM (finding 33).

18.4 Termination

Rewriting cycles by one mechanism, recorded for $\mathsf{sum_invs_const}$ of the database and met four times in the folder (findings 19, 24, 29): an open all side condition is matched against the all facts, which builds the simplifier's context of each fact's combinator and rewrites the reads $\mathsf{earlier}(\mathsf{natural}(k), \lfloor X \rfloor)$ of its dependencies there; if the left-hand side of the rule being decided matches one of those rewritten reads, the rule recurs through the same fact one level deeper until the step budget is exhausted. Two ingredients, both needed: an all condition over lists that the left-hand side binds, and a left-hand side general enough to match inside that context. Hence the rule of thumb for a rule with all conditions: its left-hand side must be a term that no rewriting of a read can produce, an all fragment with a specific body or an atom of the specification, never a bare read, comparison, equation, or $\mathsf{current}$ of a named trace for which an $\mathsf{earlier}$ acceleration lemma exists or may be added. The general substitution lemmas of the first version of `row_sums.dfy` (a higher-order body $t = F(k, J)$, a read $\mathsf{earlier}(\mathsf{natural}(i), ts)$) are exactly what this forbids (Section 3); the exit lemma of the module `nested` with the body $n \leq j$ cycled with each counter lemma and with none alone, found by pair runs, and is stated on the constraint fragment $\mathsf{integer}(nn) \leq n$ now. The order of the modules also decides which abstraction a fragment gets: $\mathsf{consts_prefix_def}$ of `aggregation` abstracts the exit $j = n$ before `nested` sees it, hence $\mathsf{consts_prefix_integer}$ with the atom as left-hand side (finding 23).

18.5 Diagnostics

The obstacles of this folder were located by four techniques, each worth a flag of the tool:

- **Preludes.** A program without lemmas and a file of lemmas spliced in front of it by the script `with.sh` of `pending/`, so that variants of the lemmas run against the same program; the folder holds the series for the cocktail sort (`cock2` to `cock9b`), the first duplicate (`fd4` to `fd8`), the lower bound, the row sums and the transpose, and the bisection of the module `nested`.
- **Truncated copies.** A summary with its first k side conditions, k downwards, finds the first failing condition (the 21st of the cocktail summary); the copies are unsound and serve the

operator	tests	MonoCera	Cuvée `-teq`	Cuvée `-chc`	TriCera, CPAchecker, SeaHorn
forall	93	26	53	0	0, 1, 0
max	4	2	2	0	0, 0, 0
min	11	3	7	0	0, 0, 0
sum	17	10	17	0	0, 0, 0
total	125	41	79	0	0, 1, 0

Table 2: Proved tests of the MonoCera evaluation (Cuvée 0.40.0, 15 min, Oct 12, 2026; the other tools' numbers from the paper). All proofs of `-teq` end within 20 s (median 9 s); no tool gives a wrong result.

diagnosis only.

- **Probes.** Rules with the real left-hand side, the first k conditions and a fresh atom $\mathsf{probe}_k(\ldots)$ as right-hand side show in the goal the largest k whose conditions hold, and a probe that fires with "variables of a rule not instantiated" shows a left-hand side that left variables unbound (finding 32). A flag `-probe <lemma>` that reports the first failing condition of a named rule, with the facts it was matched against, would replace an afternoon of probe files.
- **Pair runs.** The single lemmas of a module in pairs on the program that cycles find the pair that does (`pending/nested_module/pairs.sh`); the rule reported at the step limit is the last step of the loop, not its cause.

With `-rules` a proof reports the lemmas it used; a residual goal points to the missing one. The whole folder runs in about five minutes on one core, 5 to 35 s per file, the row sums and the transpose in under two minutes.

18.6 The same principles on C programs: the MonoCera tests

The 125 SV-COMP tests of MonoCera (Amilon et al., CAV 2023; instrumentation of $\forall$, max, min and $\sum$ over arrays) are C programs with a loop that fills an array and a loop that checks an assertion over it. Their summary (`benchmarks/monocera/summary.md`) and the case studies of the lemmas written for them (`lemmas.tex` there, Cuvée 0.35.15) show the principles of this folder on another class of programs, without hand-written specifications.

- **The shape of the data decides the lemma, not the idea.** An accumulator kept in the cell `sum[0]` of a one-element array is a trace of stores, which current_trace_reset_add of the database, written for a scalar, does not match; select_current_trace_reset_add is the same lemma for the read of the cell, with the scalar lemma's right-hand side so that the database continues from there. This is "name the normal form" (Section 18.1): the Dafny translations of SV-COMP make the cell a scalar and close the test for free, the C programs need the variant.
- **Left-hand sides bind, side conditions must be facts.** sum_cases_below bounds a sum of n terms that are each 0, c or d by e with its left-hand side a comparison whose constants bind the bound; the general sum_below_prefix of the database does not apply, since its bound occurs only in a side condition, which would have to be among the facts. The counting sort's rank_table_def binds the key bound kk for the same reason (Section 9), and the pattern $s = m + n$ of the merge is its arithmetic form.
- **Lemmas compute values, they do not make goals vacuous.** Each closed test was rerun with a false assertion, which stays open with a residual such as $\neg(0 < N)$: the lemma computed $N - 1$ and the bound was wrong. The same check as the wrong postconditions of Section 18.3.
- **Specific lemmas are summaries.** The strided stores (select_current_trace_stride2, stride 3, three exhaustive cases at stride 2) and the stores ahead ($F(k)$ at $k + 1$, then $G(k)$ at k) fix the number of stores and of cases on their left-hand sides, as the summaries of this folder fix the fragment structure of one algorithm; strides 3 to 5 with per-cell cases, writes by inner loops

and recurrences on earlier entries remain open. Both want what the arity variants perm_steps, perm_steps_two and sorted_prefix_steps_two want: a pattern with the stride, the number of dependencies, or the number of cases as a parameter.

- **The checking loop is an instantiation at a witness.** In 72 of the 96 open goals of the earlier run the assertion of the checking loop survives as an all over its earlier iterations, and the goal is the assertion at the current index $i = \mathsf{integer}(n)$ of a filled array whose closed form is not derived (recurrences $b[i] = b[i-1] + a[i-1]$, parity, division): the step from the constraint of one loop to a position of another is the one that not_occurs_selects_def and select_not_occurs_selects make for the linear search (Section 1), an atom and its instantiation, and the missing closed forms are the partial descriptions of Section 7 of the paper (the max and min tests, the lower bound here).
- **The higher-order variable and the array.** The simplifier's eta reduction turned the instance $(z) \mapsto a[z]$ of a higher-order summand G into the array a, leaving an application $a(k)$ that no rule about $a[k]$ matches; restricting eta reduction to lambdas of array type fixed it in 0.35.15. The examples of this folder meet the same phenomenon in the matcher (finding 10) and avoid a higher-order variable for a bare read, reading the array literally (`prefix_sums.dfy`) or pinning the stored values (Section 7).
- **A match that fails for no visible reason is usually one of the known patterns.** The lemma for the stores ahead of a loop that runs downwards had the same printed form as the trace in the goal and did not match, and the summary of the MonoCera tests had excluded the literal -1 as the cause. Re-traced here (Section 16), it is two of the patterns of Section 18.2 at once: the literal step -1 of the counter dependency is not matched (a variable d with $d = -1$ as a side condition is, as in `reverse.dfy`), and the position $i - 1$ of the store ahead is arithmetic on a bound variable (the pin). With both, the lemma fires and the two `rewnifrev` tests close (`rewnifrev2` stores ahead upwards in a loop that runs downwards, another shape). The cocktail summary that fired once in a day (finding 12) and the lemma of the module `nested` that fires in the input but not from the module (finding 34) are residues of the same kind, for which the probe flag of Section 18.5 is the remedy.
- **Lookup against inference.** `-chc` with the Horn solver of z3 proves none of the 125 tests (120 timeouts): the quantified invariants over arrays that the tests need, $b[k] = k^2 + k + 1$ for all $k < i$, are not found, as for TriCera in the paper, and the 15 minutes add nothing to either MonoCera or `-teq`, whose proofs end within 20 s. This is the paper's thesis in numbers: a lemma found once serves every loop of its shape, and the price is that a loop without a lemma stays open (43 of the 125, mostly the recurrences, the parity and the nondeterministic input arrays).

18.7 What the engine and the database need

The workarounds of Section 18.2 are each a missing feature of the engine, ordered by the payoff that the examples measure: a re-entrance guard, such that a rule is not applied while the instances of its own side conditions are simplified (the sound lower bound closes, the summaries lose the current(results(...)) detour); closed all conditions matched modulo unused dependencies instead of decided (the fresh variables and identifying equations disappear from five files, the summaries shrink by a third); a left-hand side matched through the definition of a trace variable binds all its variables or is refused; a static check that rejects a left-hand side with a bare bound variable of a variable dependency together with an all condition (it would have refused the cycling exit lemma on load); and a case split of `-split` on a disjunctive negated exit. For the database: the module `nested` into `-teq` with a run on the SV-COMP translations; the chain of the row sums generalized into it (counters over constant numbers of iterations, final values given per iteration, pointwise definitions as maps, reads of a map over the counter); the module `sorting`, which no file includes yet, taken up by the examples; Lean proofs for `nested`, `sorting` and `strides`, with definitions of the abstract predicates; and lemma schemas, one per observer of a named scan, in place of the arity and type variants. The algorithms that remain open need, respectively, the substitution of a solved exit

into a function argument (`is_sorted.dfy`), a witness for an existential postcondition or quantified left-hand sides (`max_element.dfy`), and the re-entrance guard (the lower bound).

19 Conclusion: Symptoms, an Engine, and the Representation

19.1 The patterns are symptoms

Section 18 presents the twelve patterns of Table 1 as a discipline for writing lemmas. Read against their causes, they are a bill for the engine. Nine of the twelve are the matcher (specific left-hand sides, fresh variables for closed conditions, the pin, variables named apart), the context (open trace variables, the whole atom as left-hand side, arithmetic in the goal's form), or the rewriting strategy (side conditions in the forms of the included modules, no condition on the rule's own term, the exit of an outer loop as two implications); one is the representation (the fragments of inner loops that do not share facts); only the atoms for quantified hypotheses and iteration constraints are partly a design choice, what a lemma may take as a fact. The residues of the MonoCera tests sort the same way: the downward stores-ahead lemma failed on a literal step and a position computed from a bound variable (Section 16), half_even exists for a solution that is not substituted (Section 15), and a definition lemma hid a constraint by rule order. What the examples strained is not the method, lookup of loop shapes in a database of lemmas, but the engine that performs the lookup: syntactic matching modulo a fixed list of theories, first match without backtracking, a context that follows definitions only where the matcher asks, conditions simplified recursively, and normal forms that depend on the order of the rules.

19.2 Trace equation systems: an interface, not the sole carrier

A loop over variables $\bar{x}$ is, semantically, one fold over the iterations, $S = \mathsf{trace}(\bar{s}_0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ \bar{s}, k \mapsto F(\bar{s}, k))$, with the state tuple as the accumulator. A trace equation system is this fold decomposed by projection: one equation per variable, with the other components as sequences among the dependencies. The decomposition is exact, every projection of a fold over a product is a fold over the product's history, and symbolic execution computes it for free. Where the dependencies form a layering (a counter, an index over it, an array over both, a sum over the array), the decomposition is the compositional reasoning of the method: each equation meets a small lemma of the database and the results are substituted downstream, which is why the single loops of this document and the arrays of SV-COMP go through with nothing written for the program. A single fold over the state would first have to discover that layering before any lemma on a counter or a sum could apply.

Where the graph of dependencies is not a layering, the decomposition is the obstacle, and these are the hard examples. Mutually dependent variables, the pointers of the merge, the bounds of the binary search, the position and the array of gnome sort, give equations that are individually meaningless, and every lemma reconstructs the product by tying the dependency lists back to their traces with open trace variables. Inner loops become fresh sequences distributed into fragments over incomparable dependency lists, which is the whole fragments problem of Sections 2 and 4.3. Equal counters that are not identified, the injective selection of dependencies, unused dependencies dropped and later missed, and the arity variants of lemmas are bookkeeping of dependency lists that a single shared dependency would not have.

The conclusion we draw is that trace equation systems are the right interface for the lemma database and the wrong sole carrier of the goal. The state trace of a loop should be the primary object, with one counter, one dependency list $\lfloor K \rfloor, \lfloor S \rfloor$, all iteration constraints over it, and the inner loop as a value of the update; the per-variable equations are views, $X = \pi_x(S)$ with $\lfloor Y \rfloor = \mathsf{map}(\mathsf{for}\ \lfloor S \rfloor\ \mathsf{yield}\ \bar{s} \mapsto \pi_y(\bar{s}))$, derived by one projection rule whenever the update of a component mentions only some components, which is exactly when a lemma of the database can apply to it. The matcher already synthesizes dependencies as maps when a pattern dependency is a variable; this is the same mechanism made principal. The lemmas keep their form.

19.3 An engine without the symptoms

Five components, from the most local to the most far-reaching, each removing a group of patterns (`ideas/deduction-engine.md` has the details and an order of work):

1. **Congruence closure as the context**, in place of the chains of variable equations and the definitions followed on demand: a pattern variable matches an equivalence class, a definition is followed in every position, both orientations of an equation are available, equal counters are identified, and solutions reach function arguments. This closes `is_sorted`, retires `half_even` and `mul_eq_cancel`, and removes the open trace variables and the whole-atom left-hand sides.
2. **Canonical forms before matching**, with closed conditions looked up modulo those forms instead of re-simplified, backtracking over the selection of dependencies, rule variables in their own namespace, and abstraction applied by the engine where a pattern subterm over bound variables fails to match. No fresh variables with identifying equations, no pins, no literal -1, no arity variants.
3. **Saturation with triggers** in place of the recursive simplification of condition instances: conditional rules fire when their conditions are present, facts about terms that occur are derived forward (the bounds of named monotone traces, the body of an iteration constraint at an index in range, the value of a trace at its exit), and a disjunctive exit is split by the engine when a rule on an atom needs a literal of it. This removes the cycles, the condition on the rule's own term, the atoms for constraints, and the dependence of normal forms on the order of modules; a condition that is a formula to prove, the step of an induction lemma or a pin, is a subgoal with the parent rule suspended.
4. **Equality saturation for the terms**, the long-term form of the first three: rules add equal nodes rather than replacing, extraction picks the result, nested e-graphs per combinator body carry the iteration facts.
5. **State traces with projections**, as above, which dissolves the fragments problem and makes the lemmas for mutually dependent shapes what their summaries are today after all the tying: a lemma on a fold over a pair with the guard as a higher-order variable.

With the first three and the representation, `is_sorted` closes by congruence, the nested sums and the two minimal problems need no fragment lemmas, the merge, gnome sort and the lower bound take one lemma on a fold over a pair, the sorting summaries take the inner fold as one condition instead of seven to twenty-one, the first duplicate keeps its equivalence, the existential postcondition of `max_element` finds its witness among the terms, and the stores-ahead lemma matches downwards as it does upwards. What stays is what the method is for: the database of introduction and acceleration rules, the symbolic execution that produces the equations, and the summaries and inductions that state what an algorithm's loops compute.

AI Declaration

The theory of trace equation systems, their generation from loops, the rewriting strategy and the matching of traces are the work of the author and his coauthors in *Trace Equation Systems for Program Verification*; the implementation in Cuvée is described in the competition contribution of Cuvée to SV-COMP 2027 (`doc/paper`), with its own declaration. The examples of `examples/dafny/algorithms`, their specifications and lemmas, the modules `nested` and `sorting` of the lemma database, the findings of `NOTES.md`, the experiments kept in `pending/`, the lemmas for the MonoCera tests and this document were produced by coding agents (Claude Code with Claude Opus; the restructuring of the document, Sections 12 and 18 with Claude Fable 5.1) between October 9 and 12, 2026, under the direction of the author, who chose the problems, set the rules of the work (no changes to the engine, no unfolding of trace combinators in lemmas for programs, every result checked on a wrong postcondition, runs with their exit status), decided the open questions, and reviewed the results. The runs reported here were made by the agents with the tool; the lemmas of the examples are axioms that no proof checks (Section 18.3), and the text should be read as the agents' account of the experiments, reviewed by the author.